

Thesis

Vehicle-in-the-Loop simulation using CARLA simulator

Author:

Kamolov Makhmud



Budapest University of Technology and Economics
Department of Control for Transportation and Vehicle Systems

Supervisor:

Szőke László

Thesis:

May 10, 2026

Contents

1 Introduction

1.1 What is autonomous driving	3
1.2 Importance of Simulation in Autonomous Driving	4
1.3 State of the Art in Autonomous Driving Simulators	6
1.4 ADAS Applications	10
1.5 Related Works	13

2 Interface Co-Simulation Design

2.1 Selecting the Suitable Simulator	16
2.2 Modeling the Car in the Environment	16
2.3 Data Gathering with Autonomous Driving Mode	21
2.4 Data Gathering with Manual Driving Mode	21

3 Lane Keeping Assist: Theory, Architectures and Method Comparison

3.1 Overview of the Three Architectures	24
3.2 The Kinematic Bicycle Model Revisited	25
3.3 The PID Feedback Law	27
3.4 Cross-Track Error Formulation	29
3.5 Heading-Error Formulation	31
3.6 System Identification and Analytical Tuning	35
3.7 Vision-Based Lane Reference	38
3.8 Method Comparison	41

A Carla and Pycharm IDE Interfacing

B Code Excerpts

BA Data Collectors (Chapter 2)	50
BB Cross-Track Error (Chapter 3, Section 3.4)	52
BC Heading Error and Look-Ahead Target (Chapter 3, Section 3.5)	53
BD PID Controllers (Chapter 3, Section 3.3)	54
BE Inverse Perspective Mapping (Chapter 3, Section 3.7)	56
BF System Identification (Chapter 3, Section 3.6)	57
BG Manual-Override Class	59

Abstract

Abstract will be added after completion of the thesis.

Chapter 1

Introduction

1.1 What is autonomous driving

For 3 million years human being has been creating tools to ease their daily chores in order to make better life. One of the successful creation of human as decent invention is vehicles. First automobile ever created is involved to late 19th century. Since that vehicles have been evolving rapidly, so, recent decades we had new concept of vehicle so-called “Autonomous Vehicle”.

Autonomous Vehicles (AVs), or self-driving cars, are vehicles as discussed earlier that can ease human daily driving tasks by taking them one by one to do it themselves.

Autonomous vehicles represent the next major step in the evolution of road transportation, extending earlier advances in vehicle automation and driver assistance systems. As sensing, computing power, and control technologies have matured, vehicles have gained the ability to monitor their surroundings, interpret complex traffic situations, and support or even replace human driving tasks. Early prototypes in the 1980s and 1990s, enabled by progress in computer vision, robotics, and embedded systems, demonstrated that vehicles could follow lanes, maintain speed, and avoid obstacles under constrained conditions. The parallel development of GPS-based positioning, digital mapping, and onboard sensors such as cameras, radars, and lidars provided the foundation for modern perception systems. Last 7 decades, Autonomous driving has got big advances which can be seen in Figure 1.1. These technological advances collectively enabled the transition from basic driver assistance toward higher levels of automation, where vehicles can execute steering, acceleration, and braking tasks with increasing independence from human input. Driverless vehicle technology gained momentum at the beginning of the 21st century. Initiatives like Google’s driverless car project [1] captured public attention and created a general awareness that autonomous driving was a real possibility. The use of cameras, radars, lidars, and artificial intelligence algorithms in vehicles endowed them with the capability to navigate complex traffic situations independently. Many car manufacturers and technology companies are now accelerating their efforts to develop fully autonomous vehicles. The goal of these vehicles is to reduce traffic accidents, increase transportation efficiency, and provide greater independence for people with mobility limitations. However, technical challenges, as well as ethical, legal, and safety issues, continue to shape the advancement in this field. The future of autonomous driving depends on overcoming these challenges and gaining societal acceptance for this new technology.

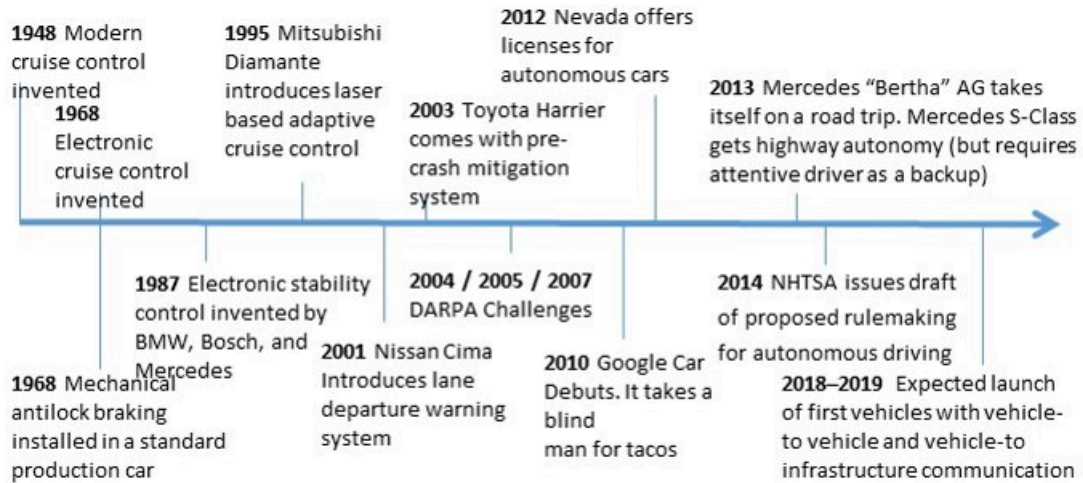


Figure 1.1: The historical development of autonomous driving [2].

1.2 Importance of Simulation in Autonomous Driving

In today's world, developing and using software has become very important across many industries. While these software solutions provide us with numerous conveniences and advantages, they also introduce certain requirements. One of the most critical requirements for any software is testing. The testing phase can be used to determine whether an application fulfills its intended function or to assess its efficiency. Of course, in some sectors, this functionality takes on an even greater importance. One such area is safety. In industries like aviation and automotive, where a user's safety can be directly affected by an error in the application, testing and simulation are more important than ever. In model-based software applications, after a model is created, it moves to the verification stage. Before being loaded onto the hardware, the model undergoes several verification steps (as V model in Figure 1.2). Some of these verifications are named Model In the Loop (MIL) [3], Software In the Loop (SIL) [4], and Hardware In the Loop (HIL) [5].

Model in the Loop (MIL) In the initial stages of system design, before real hardware or software components are implemented, the simulation and testing of system models are carried out by the Model in the Loop (MIL) testing method. This method uses mathematical models and simulations for system or component design. These models are utilized to understand how the designed system will function and behave. MIL testing is a critical tool for verifying the design and identifying potential problems at an early stage. With this method, engineers and designers can gain valuable insights into how the system will operate before actually manufacturing the hardware or fully developing the software.

Software in the Loop (SIL) The Software in the Loop (SIL) testing method is the process of testing a system or component's software without using real hardware.

This method simulates environmental factors or other parts interacting with the system while the software runs directly on a computer. This simulation evaluates the software's behavior and performance under real-world conditions. The main goal of SIL testing is to verify the software's functionality, detect errors at an early stage, and analyze interactions between the system and the software. This approach accelerates the software development and refinement processes and prevents costly errors.

Hardware in the Loop (HIL) The Hardware in the Loop (HIL) testing method creates an environment to control and test real hardware. In this method, the hardware being tested, such as a vehicle control unit, operates in real-time, but the software simulates the physical systems (e.g., engines, sensors, actuators, etc.) interacting with the hardware in real-time. This simulation allows for the evaluation of how the hardware performs under real-world conditions.

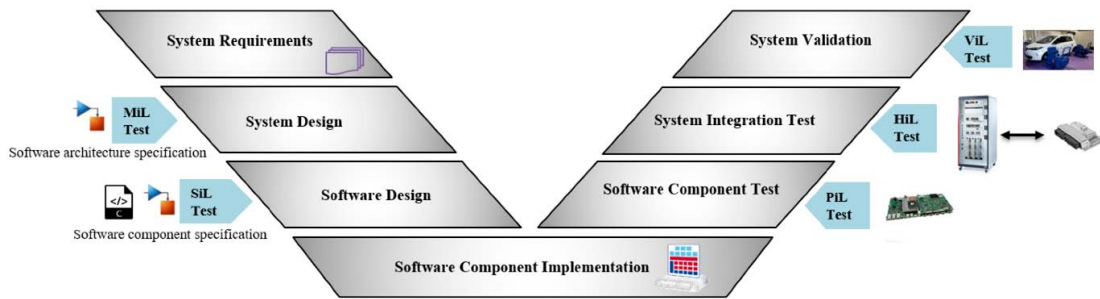


Figure 1.2: MIL, SIL , PIL, HIL and VIL tests in V-cycle development process [6].

In Figure 1.3 driving test simulator is shown. Test simulators typically fall under the SIL category. The Software in the Loop (SIL) testing method has numerous advantages. These benefits explain why SIL tests are an important part of the software development process and demonstrate their value to engineers and developers across various industries. The main advantages can be listed as follows:

Cost Efficiency: SIL tests are conducted in a simulated environment that does not require real hardware, thus reducing the costs associated with purchasing, maintaining, and repairing hardware. Additionally, early diagnosis of potential errors helps prevent more costly problems later on.

Rapid Feedback Loop: Testing software in a model allows for quick iterations in the development process. This enables developers to instantly see the effects of their code and make fast modifications if necessary.

Risk Reduction: Tests conducted without real hardware use a simulated environment to safely examine situations that could be potentially dangerous or harmful to the hardware. This is especially important when expensive hardware is involved.

Broad Test Scenarios: SIL tests are capable of quickly and easily testing a wide range of scenarios that mimic real-world conditions. Developers can experiment with system parameters, error states, and various operational conditions.

Acceleration of the Development Process: SIL tests can speed up the development process instead of waiting for the hardware to be ready. Testing the software alongside hardware development shortens the time to market for the product.

Preparation for Integration and System Tests: When software is successfully tested in a SIL environment, the transition to more complex testing stages where hardware and software are run together becomes easier. This helps reduce problems that may arise during system tests and integration.

Flexibility in the Development Process: Software can be tested and developed with various hardware platforms and configurations. This flexibility allows for the evaluation of how the software will perform on different systems.



Figure 1.3: Driving Test Simulator [7]

1.3 State of the Art in Autonomous Driving Simulators

Advancements in the field of autonomous driving have also led to the development of powerful autonomous driving simulators. Nowadays, a wide range of simulators produced by various companies are in use. These include open-source projects (e.g. CARLA [8], AirSim [9]), commercial software, and customized simulation solutions used in academic and industrial research. In the context of this thesis, a simulation environment is essential, because the Lane Keeping Assist (LKA) function is developed and evaluated entirely in a virtual Vehicle-in-the-Loop setup rather than on a physical test vehicle. Therefore, it is crucial to clearly define the requirements that the simulator must satisfy—such as realistic sensor models, controllable traffic and weather, and reproducible scenarios—and to justify the choice of a specific platform.

In the following, I first review several popular autonomous driving simulation tools and platforms available in the literature and in practice. The comparison focuses on criteria such as graphic quality, accuracy of the physics engine, sensor simulation capabilities,

simulation of traffic and pedestrians, weather conditions, and the ability to simulate at different times of the day. Based on these criteria, I motivate the selection of CARLA as the main simulation platform for this work and explain how it will be used to implement and test the proposed LKA function.

1.3.1 Waymo simulator



Figure 1.4: Comparing Waymo Simulator graphics and the real world.

Waymo possesses an advanced simulation platform [10] and is widely recognized as a leader in autonomous vehicle technology. This simulator plays a crucial role in the development and testing of autonomous driving systems by providing a highly realistic (can be seen in Figure 1.4) and comprehensive virtual environment. It combines high-fidelity graphics that accurately replicate real-world environments with a precise physics engine capable of modeling vehicle dynamics, collisions, and surface interactions. Furthermore, it includes detailed sensor emulation for LIDAR, radar, and cameras, enabling a realistic representation of environmental perception. The platform can also model complex traffic flow and pedestrian behaviors, which are essential for evaluating autonomous vehicles in dense urban settings. In addition, the simulator supports a wide range of weather and lighting conditions, allowing performance assessment under rain, snow, fog, or low-light scenarios. Owing to these capabilities, the Waymo simulator holds a significant position among state-of-the-art autonomous driving simulation platforms.

1.3.2 SVL Simulator



Figure 1.5: SVL Simulator Screen

The autonomous vehicle research community frequently employs the LGSVL Simulator [11], an open-source platform designed for high-fidelity testing of perception and control algorithms. Thanks to its Unity-based rendering engine, LGSVL offers realistic visual environments suitable for evaluating camera-based functions. Its physics engine provides detailed modeling of vehicle dynamics, tire-road interactions, and collisions, enabling credible closed-loop testing of control algorithms. The simulator includes configurable sensor models—such as lidar, radar, and cameras—along with comprehensive traffic and pedestrian behavior, allowing researchers to model complex urban scenarios (Figure 1.5). Furthermore, LGSVL supports diverse weather and lighting conditions, which makes it valuable for assessing the robustness of autonomous driving functions under varying environmental circumstances. LGSVL represents an important reference point when comparing available simulation platforms.

1.3.3 Sim4CV

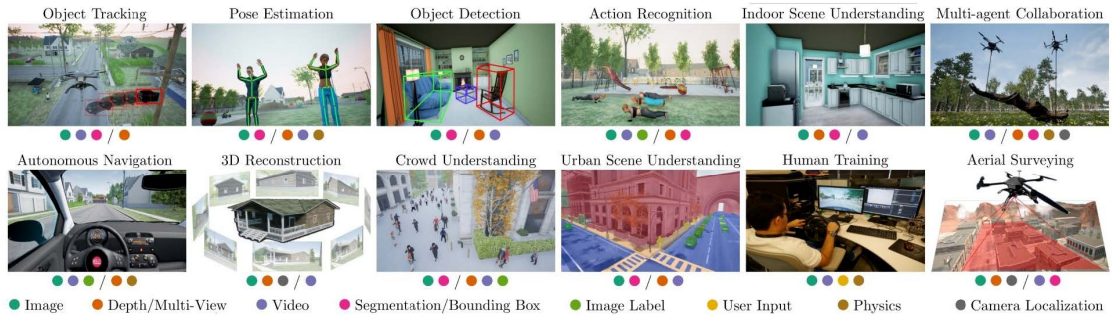


Figure 1.6: Sim4CV: A Photo-Realistic Simulator for Computer Vision Applications

Sim4CV is a simulation platform developed for research in computer vision and autonomous systems, with a strong emphasis on autonomous driving and aerial navigation [12]. Built on Unreal Engine 4, it provides high-quality visual environments and a

physics engine capable of modeling realistic vehicle dynamics, object interactions, and environmental effects. The platform supports the simulation of various sensors, including cameras and lidar, enabling detailed testing of perception algorithms. Sim4CV also offers traffic and pedestrian simulation, though the level of sophistication may vary across versions and configurations (Figure 1.6). Furthermore, the ability to adjust weather and lighting conditions makes it suitable for evaluating algorithm robustness under diverse environmental scenarios.

1.3.4 Carla Simulator



Figure 1.7: Carla Simulator Screen

The CARLA Simulator is an open-source autonomous driving platform [13] widely used in research for developing, testing, and validating autonomous vehicle algorithms. Built on Unreal Engine 4, it provides high-fidelity graphics that closely resemble real-world environments (Figure 1.7), which is essential for evaluating perception systems. CARLA’s underlying physics engine accurately models vehicle dynamics, collisions, and surface interactions, enabling realistic assessment of control and planning algorithms. It also offers comprehensive sensor simulation—including cameras, LIDAR, radar, and GNSS—allowing researchers to replicate and analyze diverse sensing conditions. In addition, CARLA supports dynamic traffic scenarios and human-like pedestrian behavior, creating complex and socially interactive environments for autonomous driving studies. Finally, its configurable weather and lighting system makes it possible to test algorithms across various environmental conditions, ensuring robustness and generalizability in real-world deployments.

Features / Simulators	Waymo Simulator	LGSVL Simulator	Sim4CV	CARLA Simulator
Graphics Quality	8/10	9/10	8/10	9/10
Physics Engine Accuracy	8/10	9/10	7/10	9/10
Sensor Simulation	8/10	9/10	7/10	9/10
Traffic and Pedestrian Simulation	8/10	9/10	6/10	9/10
Weather Conditions	7/10	8/10	6/10	9/10
Simulation at Different Times of Day	7/10	8/10	6/10	9/10

Table 1.1: Comparative Feature Scoring of Simulators

Using a set of predefined evaluation criteria—such as graphics quality, physics realism, sensor simulation, traffic modeling, and environmental variability—a comparative score was assigned to each simulator. Table 1.1 summarizes the evaluation results based on these criteria.

1.4 ADAS Applications

Advanced Driver Assistance Systems (ADAS) are becoming increasingly common in today’s vehicles. Developed to enhance vehicle safety and the driving experience, these systems utilize various technologies such as cameras, sensors, and radars to identify potential hazards and are designed to alert the driver or automatically control the vehicle under certain conditions. ADAS technologies not only assist drivers in traveling more safely and comfortably but also have the potential to reduce traffic accidents. There are various applications of ADAS, which are crucial for the safety of the driver and also provide significant conveniences. Some of these applications can be briefly discussed as follows:

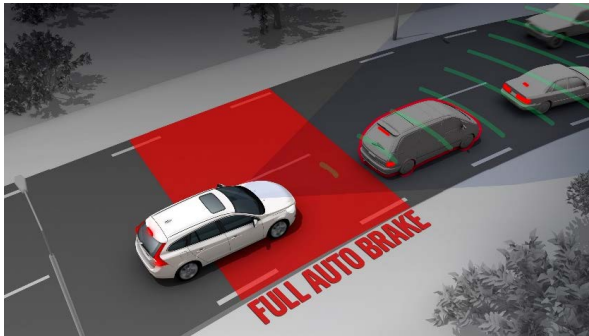


Figure 1.8: Automatic Emergency Braking system.

Automatic Emergency Braking (AEB):

The vehicle automatically brakes to avoid colliding with a vehicle or obstacles ahead [14]. A laser radar sensor detects the distance to the vehicle ahead and the relative speed. If there is an object in the area seen by the laser radar sensor at this speed, the brakes are activated to prevent a collision as shown in Figure 1.8. Additionally, in some vehicles, it works in conjunction with seat belts activated by braking before a

collision to help reduce injuries in cases where a collision is unavoidable.

Traffic Sign Recognition System: This system detects traffic signs and informs the driver about traffic rules such as speed limits and prohibition signs. Generally, an image is captured by a camera sensor placed on the vehicle’s windshield, and the detected image is projected onto the user’s screen (Figure 1.9). This technology aims to prevent accidents by ensuring that drivers do not overlook important warning signs on the road [15].



Figure 1.9: Traffic Sign Recognition System



Figure 1.10: Night Vision

20% of all road accidents, and up to 50% on certain roads, are fatigue-related [17]. Although the implementation of this system varies among manufacturers, some brands warn the driver by detecting steering movements and how often the vehicle drifts out of its lane.

Blind Spot Warning System: This system identifies the presence of other vehicles in the vehicle's blind spots and informs the driver about them. Vehicles in the blind spot are detected using radar sensors located on the sides of the rear bumper (Figure 1.11). A warning is displayed to the driver in the side mirror, and if the driver signals to change lanes while there is a vehicle in the blind spot, an alert is issued. In some models, changing lanes by braking is prevented to enhance safety of the driver in the car and to avoid road accident [18].

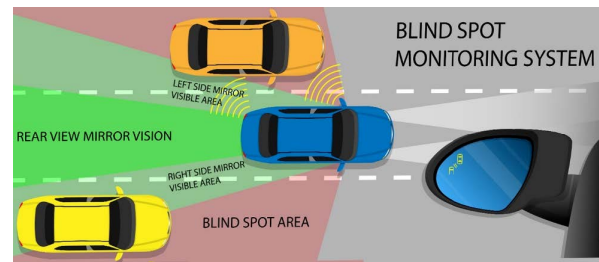


Figure 1.11: Blind Spot Warning System

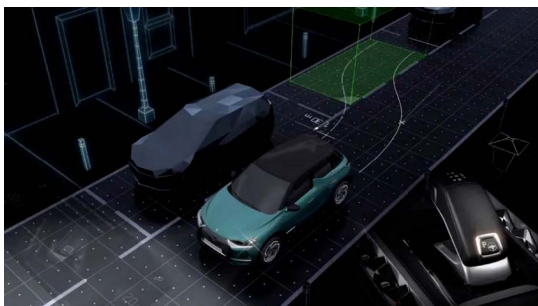


Figure 1.12: Parking Assistant

Night Vision: This system expands the driver's field of vision in low light conditions or at night through cameras or other sensors [16]. Utilizing thermal cameras and infrared lights, it detects living beings on the road ahead and provides audible or visual warnings to the driver (Figure 1.10). In addition to detection, advanced night vision systems can highlight potential hazards directly on the dashboard display or head-up display.

Fatigue Detection Systems: Various studies have suggested that approximately

Parking Assistant: This feature assists the driver in better positioning the vehicle while parking and, in some cases, can automatically park the vehicle (Figure 1.12). It utilizes parking sensors located around the vehicle to do this [19]. Parking sensors are generally electromagnetic sensors. Commonly, the system is used in a manner where the driver is still responsible for commands such as gas and gear changes.

Automatic Headlight Control: The effectiveness of vehicle headlights becomes even more crucial on roads with insufficient external lighting at night, such as forest roads. Oncoming drivers on these types of roads can be affected by the headlight beams. Systems like this reduce the light intensity when detecting an oncoming vehicle to minimize the adverse effects on the opposite driver. In more advanced models, the direction of the headlight beam is adjusted to achieve this effect (Figure 1.13).

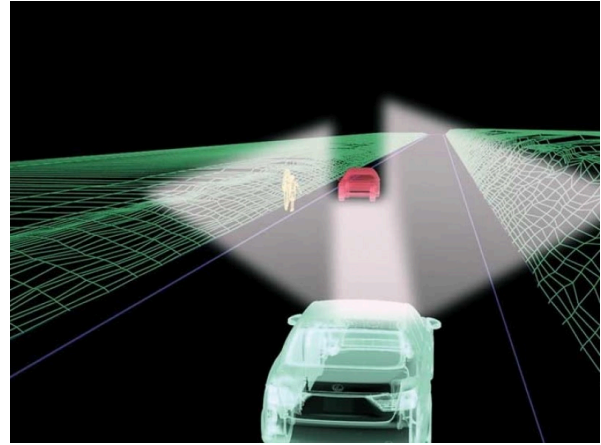


Figure 1.13: Automatic Headlight Control



Figure 1.14: Lane Keeping Assistant

Lane Keeping Assistant: This system uses a camera sensor to detect the distance of the vehicle from the lane markings and activates when the determined distance falls below a certain threshold (Figure 1.14). It steers the steering wheel to direct the vehicle back into the lane, ensuring it remains within its lane boundaries. It acts as a countermeasure to unintentional drifting caused, reducing relevant accidents by over 20%.

Adaptive Cruise Control (ACC): This system is a more advanced version of the traditional cruise control system [20]. While a traditional cruise control maintains the vehicle at a set speed, this system adapts the speed based on the speed of the vehicle ahead (Figure 1.15). It utilizes two separate sensors to accomplish this. The first sensor is a radar sensor that measures the distance to the vehicle in front. The second sensor is a speed sensor that detects any decrease or increase in the vehicle's speed. Based on the information from these sensors, decisions are made and implemented to accelerate or decelerate. Generally, the desired distance and following speed can be adjusted by the user via controls on the steering wheel.

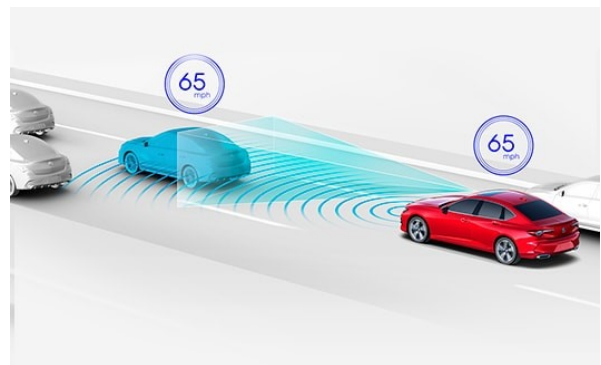


Figure 1.15: Adaptive Cruise Control

1.5 Related Works

The research community has proposed various methods for validating autonomous driving functions using simulation and Vehicle-in-the-Loop (ViL) concepts, as well as a wide range of lateral control strategies for path and lane tracking. This section summarises the most relevant works and explains how they relate to the present thesis, which implements a Lane Keeping Assist (LKA) function in the CARLA simulator using an external Python-based controller and an RGB camera.

Vehicle-in-the-Loop simulation

Son et al. propose a PG-Based Vehicle-In-the-Loop Simulation (VILS) framework that combines a real vehicle with a virtual driving environment for system development and consistency validation [21]. Their system includes modules for virtual road generation, synchronisation between real and virtual domains, a virtual traffic manager, and perception sensor modelling. They demonstrate that the behaviour observed in the VILS setup is consistent with physical vehicle tests over different speeds, road types and surrounding environments. This work shows how ViL can improve reproducibility and safety when testing autonomous driving systems.

Cheng et al. provide a comprehensive survey on testbench-based Vehicle-in-the-Loop simulation testing for autonomous vehicles [22]. They review existing autonomous vehicle testing approaches and identify limitations of traditional methods. The paper then focuses on testbench-based VIL architectures, principles and equipment, with particular emphasis on physical signal stimulation and realistic road condition simulation. The survey identifies research gaps between current testbench technologies and industrial AV testing needs, and highlights ViL as a promising tool for accurate and efficient validation.

Xiong Hu et al. present a ViL simulation test framework for autonomous vehicles, where an autonomous driving system is evaluated using a virtual environment combined with real vehicle components [23]. Their work emphasises the synchronisation of vehicle dynamics and simulated scenarios and discusses how ViL can be used to test autonomous driving functions safely and systematically.

These works clearly show that ViL is an important methodology for validating autonomous driving systems. However, they mainly focus on general architectures, testbench setups, and system-level validation. They do not specifically address a software-only ViL configuration in a driving simulator like CARLA, where a simulated vehicle is controlled by an external code using RGB camera images. Furthermore, the detailed design and evaluation of a single ADAS function such as Lane Keeping Assist—implemented and tested entirely in such a ViL setup—remains less explored. This is where the present thesis contributes.

Lateral control for autonomous vehicles

Lateral control is a key component for lane keeping and path tracking. Kebbaty et al. provide a technical review of lateral control methods for autonomous wheeled vehicles

[24]. They classify control approaches into model-based and data-driven methods, and discuss their advantages, limitations and implementation challenges. The survey underlines that lateral control must deal with accuracy, robustness and real-time constraints, and highlights open challenges for future research.

Artuñedo et al. present a comparative evaluation of state-of-the-art lateral control strategies for autonomous vehicles [25]. They propose a systematic tuning methodology and define a comprehensive set of performance metrics covering tracking accuracy, robustness and comfort. The controllers are evaluated in extensive simulations and real-vehicle tests over different trajectories and scenarios. Their results show how different control strategies behave under the same conditions and provide guidance for selecting suitable lateral controllers.

Dominguez et al. experimentally compare several classical lateral controllers—such as Pure Pursuit and Stanley—on an autonomous vehicle platform [26]. They study the performance of these controllers in terms of tracking error and stability, using a combination of simulation and real-world experiments. Their work provides practical insight into the behaviour of commonly used path-tracking controllers and their suitability for autonomous driving applications.

Snider’s technical report is a widely cited reference on automatic steering methods for autonomous automobile path tracking [27]. It describes the mathematical formulation and implementation details of several steering controllers, including Pure Pursuit and Stanley, and discusses their application to autonomous vehicles. This report is frequently used as a theoretical foundation when implementing classical lateral control laws.

These studies provide a solid theoretical and experimental basis for the lateral control part of Lane Keeping Assist. However, most of them either assume ideal path or lane information (for example, predefined waypoints or ground-truth lane geometry) or focus on real-vehicle experiments without an explicit simulation-based ViL framework. They do not analyse in detail how classical lateral controllers perform when they receive lane information from a real-time camera-based perception pipeline inside a simulator.

Identified gap and relation to this thesis

In summary, the literature on ViL shows how combined real–virtual setups can be used to validate autonomous driving systems [21],[22],[23]}, while the lateral control literature provides a variety of controllers and comparative evaluations for path and lane tracking [24],[25],[26],[27]}. However, the combination of these two aspects

- a **Vehicle-in-the-Loop-style simulation environment** in a high-fidelity simulator, and
- a **camera-based Lane Keeping Assist function** implemented as an external control stack — is less represented in the existing works.

The present thesis addresses this gap by implementing a modular LKA system in CARLA, where an RGB camera provides input to an external Python program that estimates lane geometry and computes steering commands. The system is evaluated in a reproducible simulation environment, thus connecting the ideas of ViL-based validation and classical lateral control within a single, integrated Lane Keeping Assist framework.

Chapter 2

Interface Co-Simulation Design

2.1 Selecting the Suitable Simulator

The development and testing of autonomous driving technologies require a robust simulation environment. This environment must accurately model the real world, including vehicles, pedestrians, and various environmental conditions, while also providing comprehensive support for sensor simulation and enabling integration with analytical tools. After a detailed evaluation of existing simulators, including the Waymo Simulator, LGSVL Simulator, Sim4CV, and CARLA Simulator, based on critical features such as graphic quality, the accuracy of the physics engine, sensor simulation capabilities, the simulation of traffic and pedestrians, weather conditions, the ability to simulate different times of day, CARLA Simulator has been identified as the most suitable choice for my research objectives. CARLA provides high-quality graphics with Unreal Engine 4 [28] and its physics engine accurately models vehicle dynamics and environmental interactions, offering a solid foundation for testing autonomous driving algorithms under various conditions. Moreover, its ability to simulate a wide range of sensors used in autonomous vehicles, such as cameras, LIDAR, radar, and GNSS, with high fidelity is crucial for the development and testing of perception algorithms. CARLA also excels in simulating dynamic traffic scenarios and pedestrian behaviors, facilitating comprehensive testing of autonomous driving systems in complex urban environments. The capability to simulate different weather conditions and times of day is important for assessing the performance of autonomous vehicle systems under various environmental conditions. For my research, CARLA's Python API facilitates easy integration with an external controller, providing a seamless workflow for data analysis and algorithm testing. Consequently, CARLA Simulator's advanced graphics, accurate physics engine, extensive sensor simulation capabilities, and effective traffic and pedestrian simulation set it as the ideal choice for my autonomous driving research. Its compatibility with Python API further supports my analytical and development needs, making it the most suitable simulator for my project.

2.2 Modeling the Car in the Environment

In order to control a vehicle in the simulation, the controller must recognize the vehicle model. For this, a model of the vehicle is needed. Various models can be used when modeling the vehicle. Among these, the most commonly used ones usually offer a good

balance between simplicity and accuracy, capable of representing the vehicle's motion dynamics and control systems. In line with the needs of this project, the Dynamic Single-Track (DST) model has been chosen (shown in Figure 2.1). The DST model (bicycle model) can be modeled with single-track wheels (one front and one rear wheel). This is equivalent to a model where the right and left sides of a four-wheeled vehicle are considered equal.

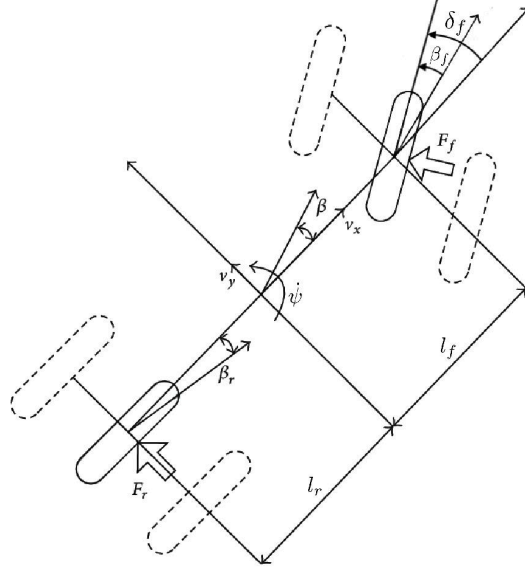


Figure 2.1: Single Track Model

Vehicle variables:

- δ_f : steering angle.
- β : vehicle slip angle = angle between the vehicle longitudinal axis and velocity
- β_f, β_r : tire slip angles = angles between the tire longitudinal axis and velocity.

Vehicle parameters:

- CoG: center of gravity
- m, J : mass and moment of inertia
- l_f : distance CoG - front wheel center
- l_r : distance CoG - rear wheel center
- c_f, c_r : front/rear cornering stiffnesses.

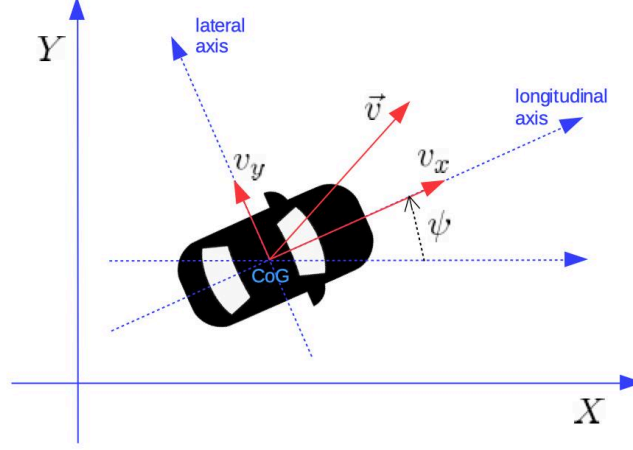


Figure 2.2: Vehicle Reference System

Vehicle Dynamic parameters:

- X, Y : coordinates of the vehicle CoG in an inertial reference frame (shown in Figure 2.2)
- ψ : yaw angle
- $\omega_\psi = \dot{\psi}$: yaw rate
- $\vec{v} \equiv V$: velocity vector in the inertial frame
- v_x : longitudinal speed = $\vec{v}$ component along the longitudinal axis
- v_y : lateral speed = $\vec{v}$ component along the lateral (transverse) axis
- a_x : longitudinal acceleration in the inertial frame.

The state equations of the DST model are:

$$\dot{X} = V_x \cos \psi - V_y \sin \psi \quad (2.1)$$

$$\dot{Y} = V_x \sin \psi + V_y \cos \psi \quad (2.2)$$

$$\dot{\psi} = \omega_\psi \quad (2.3)$$

$$\dot{V}_x = V_y \omega_\psi + a_x \quad (2.4)$$

$$\dot{V}_y = -V_x \omega_\psi + \frac{2}{m}(F_{yf} + F_{yr}) \quad (2.5)$$

$$\dot{\omega}_\psi = \frac{2}{J}(l_f F_{yf} - l_r F_{yr}) \quad (2.6)$$

where F_{yf} and F_{yr} are the lateral forces exchanged between tire and road.

For the tire model, there are several tire model options:

Linear (for $V_x = \text{const}$) tire model:

$$F_{yf} = -C_f \beta_f, \quad F_{yr} = -C_r \beta_r \quad (2.7)$$

$$\beta_f = \frac{V_y + l_f \omega_\psi}{V_x} - \delta_f, \quad \beta_r = \frac{V_y - l_r \omega_\psi}{V_x} \quad (2.8)$$

Nonlinear simplified tire model:

$$F_{yf} = -C_f \beta_f \cos \delta_f, \quad F_{yr} = -C_r \beta_r \quad (2.9)$$

$$\beta_f = \arctan\left(\frac{V_y + l_f \omega_\psi}{V_x}\right) - \delta_f, \quad \beta_r = \arctan\left(\frac{V_y - l_r \omega_\psi}{V_x}\right) \quad (2.10)$$

Nonlinear Pacejka's tire model:

$$F_{yf} = -f_p(\beta_f) \cos \delta_f, \quad F_{yr} = -f_p(\beta_r) \quad (2.11)$$

where β_f and β_r and $f_p(\beta)$ is given by the Pacejka's magic formula. **Pacejka's magic formula:**

$$f_p(\beta) = p_1 \sin(p_2 \arctan(p_3 \beta - p_4(p_3 \beta - \arctan(p_3 \beta)))) \quad (2.12)$$

p_1 : peak value, p_2 : shape factor, p_3 : stiffness factor, p_4 : curvature factor. Linearizing this formula, we find $p_1 p_2 p_3 = C_f$ (or C_r).

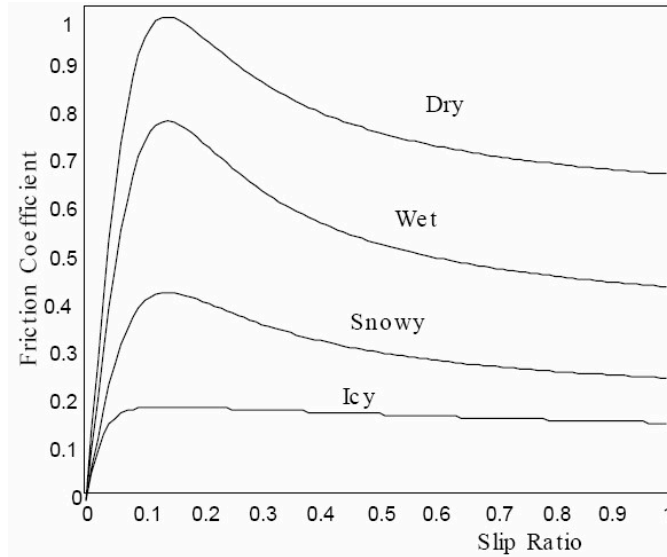


Figure 2.3: Friction Coefficients in different conditions

In the real world conditions, these parameters are really hard to measure or estimate because they change based on the road conditions (see Figure 2.3).

The real parameters of the vehicle in the simulation environment have been found with the help of Carla's `Vehicle.physics` command.

Vehicle Parameters Values

- m : 1318kg
- J : 2500kg/m²
- l_f : 1.54m
- l_r : 1.51m
- c_f : 15000N/rad
- c_r : 15000N/rad

2.2.1 Dispatching To Obtain Throttle-Brake Value From Acceleration Value

To control a vehicle in the Carla simulator, we need three pieces of information: steering angle, throttle, and brake commands. However, our model generates ax data. It is not possible to directly provide this data to Carla because this information only tells us about the acceleration or deceleration of the vehicle and the magnitude of these changes. Therefore, we need to scale this data to use it for throttle-brake information. To do this, we perform dispatching. In the context of control, “dispatching” usually refers to the process of distributing tasks or resources according to certain criteria or algorithms.

To accomplish this, we first need specific reference values. These reference values should be collected according to many different scenarios to ensure they are suitable for every scenario, not just one. In our case, data were collected for three different scenarios: when the vehicle completes a straight path in a sine wave pattern, when continuous braking and accelerating are performed on a straight road, and finally, when only accelerating is done and the vehicle slows down without braking due to its own weight. The data include the vehicle's ax acceleration value, and throttle and brake values. By examining these accelerations, the priorities of throttle and brake values are determined for acceleration and deceleration situations. For instance, it is not always necessary to brake for negative acceleration values; reducing the throttle can also achieve the necessary speed reduction. Therefore, each value has its weight. Also, there is a range within which each command should be applied. These ranges can be determined using an if/else structure based on the acceleration value. The commands to be applied within these ranges are found by dividing the ax value by certain weight values. Consequently, the throttle and

brake values are instantaneously derived from the ax value and provided to the vehicle in the Carla environment for application. As explained above, these values were found by analyzing reference data and through trial and error. The code for the if/else structure that creates the intervals for applying these values and commands is provided below.

2.3 Data Gathering with Autonomous Driving Mode

A reference point was selected from the Carla Map to collect reference data. The vehicle was spawned at this reference point. A simulation duration was determined based on a finish point we had predetermined. The vehicle was moved using Carla’s autonomous driving mode. During this time, the necessary reference position, waypoint data, yaw angle, and speed data were recorded into a `.csv` file at intervals of $T_s = 0.05$ seconds. Once the required simulation duration was completed, the vehicle was removed from the map.

Two collector routines are used during the recording. The first one, `carla_data_collector`, queries the full localization information of the vehicle at every tick: world position (x, y, z) , rotation (pitch, yaw, roll), linear velocity, linear acceleration and angular velocity. The second one, `carla_data_collector_2`, focuses on the geometry of the lane and returns, for each tick, the position of the closest waypoint together with the unit forward vector $\hat{t} = (t_x, t_y)$ tangent to the lane in the legal direction of travel. The two collectors are complementary: the first one captures the dynamic state of the ego vehicle, the second one captures the geometric reference that the lane-keeping controller will track. The complete listings of both routines are reported in Section B for completeness; here we only describe the meaning of the recorded fields.

In order to keep the resulting `.csv` file compact, the second collector is wrapped by a duplicate-suppression block. The position of the closest waypoint can remain unchanged for several consecutive ticks—for instance when the vehicle stops at a traffic light—and writing identical rows to the file would simply increase its size without adding information. The block compares each new sample with the last one written and appends it only if at least one of the two coordinates has changed. The resulting file therefore contains a strictly monotonic sampling of the centre line, indexed by the order in which the waypoints were visited.

2.4 Data Gathering with Manual Driving Mode

The first capability that we demonstrate with CARLA is localization, which allows our ego-vehicle to determine its pose in the world. Two coordinate frames: the map frame, which is a coordinate frame that is fixed at the initial position of the map, and the vehicle frame, which is a coordinate frame attached to the middle of the rear axle of

the vehicle. For our particular experiment, we record the vehicle’s accurate pose while traversing a curved-straight route in the Town10. In the Python API, there is a class that comprises all the localization information for an actor at a certain moment in time; its methods comprise Getters such as `get_acceleration`, `get_velocity`, `get_transform`, and `get_angular_velocity`- which in our case we utilize the `get_transform` method which includes both location of the object (X , Y , Z from the origin of the map) in meters, and its rotation characteristics from which we utilize the yaw values. More concisely, X and Y coordinates were our focus, as in the map chosen, the road is entirely flat. Hence only these two coordinates remain crucial for tracking the vehicle’s trajectory. Also, the Yaw angle, describing the orientation of the map’s coordinate system, is essential for indication of the direction the vehicle is facing. Therefore for curved paths, which our scenarios contain, it is important.

Some additional considerations also are to be noted, such as the sampling rate, which determines the time step of the data collection. We have chosen 0.1, indicating not too sparse to miss some critical dynamics, especially for sharp turns, and also not too frequent leading to redundant information.

After initializing the scenario and ego vehicle (based on the preferences), based on the duration of the scenario depending on the controls of the car (throttle, brake, and steering commands), we record the data of the vehicle at each time step. The corresponding logging routine has the same overall structure as in the autonomous mode—at every simulation tick the snapshot frame, the pose, the velocity, the acceleration and the applied control are appended to a Pandas dictionary which is finally written to disk as a `.csv` file. The full code excerpt is provided in Section B.

In manual driving mode, the autopilot is disengaged with `vehicle.set_autopilot(False)` and the user is given full control of throttle, brake, and steering through the keyboard. Each pressed key is captured as a PyGame event and is translated into a `carla.VehicleControl` object that is then applied to the ego vehicle in the simulator. While the user drives, the same getters introduced for the autonomous mode are queried at every simulation tick, so that the resulting `.csv` file shares the same structure for both modes and can be loaded by the controller without any additional preprocessing.

This dual-mode acquisition serves two purposes. First, it provides reference trajectories for the dispatcher tuning and for the controller design, since the autonomous mode of CARLA produces well-formed paths along the center of the lane. Second, it allows the recording of edge cases that are hard to reproduce automatically—for example aggressive lane changes, harsh braking, or following curves at the limit of stability—because they can be performed deliberately by a human driver. The two `.csv` files are then used as the input data set for the lane keeping assist controller described in Chapter 3.

Chapter 3

Lane Keeping Assist: Theory, Architectures and Method Comparison

The previous chapter established the simulation environment, the vehicle model and the data-gathering pipeline. This chapter develops the lateral-control component of the Lane Keeping Assist (LKA) function and presents three progressively richer architectures that have been implemented in the same CARLA framework. The discussion is deliberately kept theoretical: the goal of the chapter is to make explicit *why* each design choice was made, what the corresponding plant model looks like, and what the analytical tuning procedure predicts in terms of closed-loop behaviour. Code excerpts are gathered in Section B so that the present chapter can focus on the reasoning. The numerical evaluation of each controller in the simulator is the subject of the next chapter.

The chapter is structured as follows. Section 3.1 gives a high-level description of the three architectures and motivates the order in which they are introduced. Section 3.2 revisits the kinematic bicycle model of Section 2 from the perspective of control-oriented modelling, and derives the input-output transfer function from the steering command to the lateral state of the vehicle. Section 3.3 recalls the PID feedback law and its discrete-time implementation, with particular attention to the issues of integral wind-up and output saturation that arise in any practical realisation. Section 3.4 and Section 3.5 then specialise the general framework to the two error signals that are most commonly used in path-following applications: the cross-track distance and the heading error to a look-ahead point. The two formulations differ only in the choice of error signal, but as it will be shown, this single choice has profound consequences on the order of the resulting plant, on the type of controller that is required for stability, and on the way the gains have to be scheduled with vehicle speed. Section 3.6 presents a systematic identification procedure that fits the parameters of the plant from open-loop step responses recorded in the simulator, and derives analytical PID gains from a pole-placement criterion. Section 3.7 finally extends the heading-error formulation with a perception layer based on a deep convolutional lane-detection network and on inverse perspective mapping, which removes the dependence on a pre-recorded reference path. Section 3.8 summarises the relative merits of the three approaches from a control-theoretic standpoint.

3.1 Overview of the Three Architectures

The lateral-control function has been implemented in three increasingly sophisticated forms, each of which is studied in a dedicated module of the source tree:

- *Cross-track PID* (`lane_shift_pid/`). The lateral error is the signed perpendicular distance between the vehicle and the closest waypoint on the recorded reference path. A discrete-time PID controller produces the steering command directly from this distance.
- *Heading-error PID* (`heading_error_pid/`). The lateral error is the signed angle between the vehicle’s forward direction and the line that connects the vehicle to a look-ahead point placed on the reference path at a speed-dependent distance ahead. The error signal is now an angle, and a discrete-time PI controller is sufficient to drive it to zero.
- *Vision-based heading-error PID* (`detection&pid.py`). The reference path is no longer read from a pre-recorded `.csv` file. Instead, the centre line of the lane is reconstructed at every tick from the RGB camera using a deep convolutional lane-detection network and an inverse-perspective-mapping projection. A look-ahead target is selected on the reconstructed centre line and is fed to the same PI controller as in the previous case.

The three architectures are sketched side by side in Figure 3.1. They share the same actuator (the `carla.VehicleControl.steer` field), the same simulation rate ($T_s = 0.05$ s), the same vehicle dynamics, and—crucially—the same lateral plant up to the choice of the error signal. The differences between them are confined to two well-defined components: the way the lateral error is constructed, and the structure of the PID controller that consumes it. This modular separation makes it possible to compare the three approaches on equal footing, and to attribute the observed differences in behaviour to specific design choices rather than to incidental implementation details.

Three lateral-control architectures compared

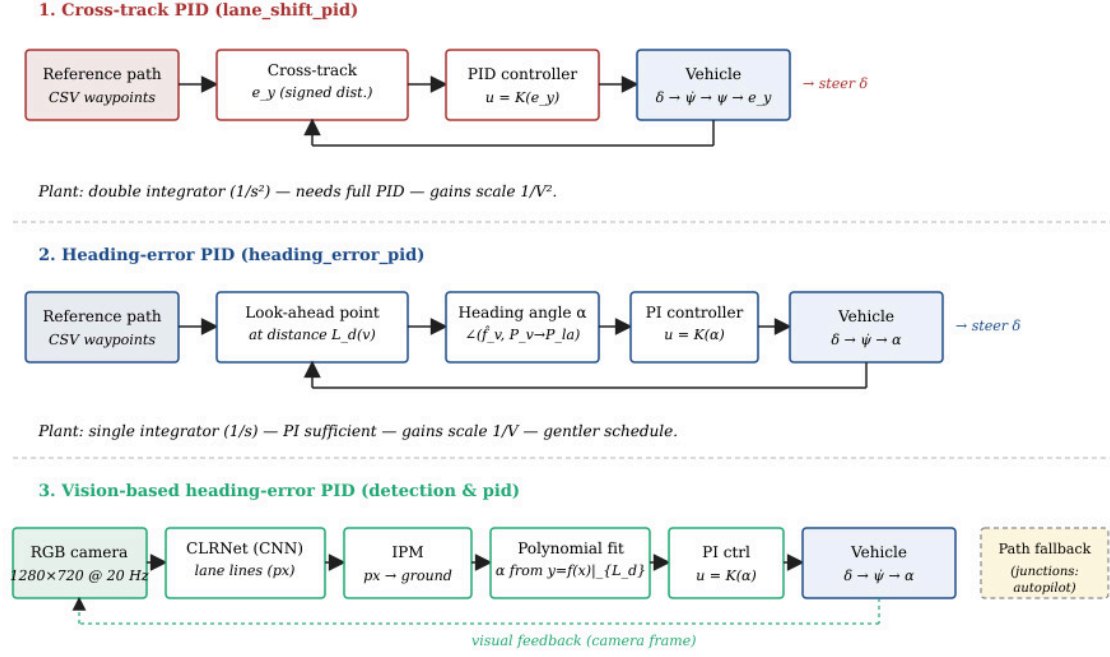


Figure 3.1: Block-diagram comparison of the three lateral-control architectures investigated in this thesis. The first one closes the loop on the cross-track distance computed against a recorded path; the second one closes the loop on a heading angle derived from a speed-dependent look-ahead point; the third one replaces the recorded path with a perception pipeline that reconstructs the centre line of the lane from the camera image. All three architectures share the same actuator, the same vehicle dynamics, and the same PID structure.

3.2 The Kinematic Bicycle Model Revisited

The kinematic bicycle model introduced in Section 2 is the simplest description of the planar motion of a four-wheeled vehicle. It collapses the front pair of wheels into a single equivalent wheel placed at the front axle, and similarly for the rear pair, and ignores tyre slip. Under these assumptions the configuration of the vehicle is fully described by three quantities: the position (x, y) of a reference point—conventionally taken at the centre of the rear axle—and the yaw angle ψ that the body of the vehicle makes with the world x -axis. The configuration space is therefore $\mathbb{R}^2 \times S^1$, and a useful sketch of its variables is given in Figure 3.2.

Kinematic bicycle model

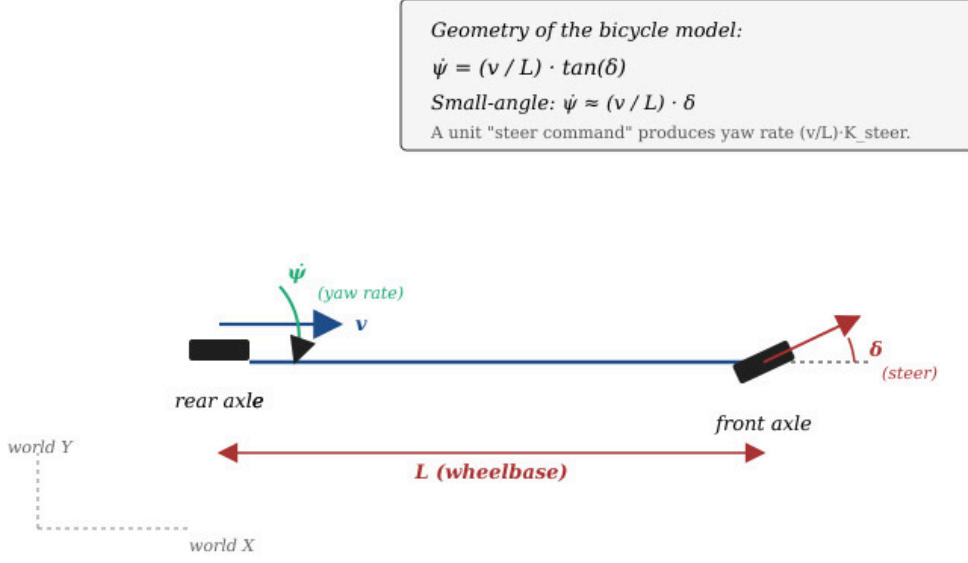


Figure 3.2: Kinematic bicycle model. The vehicle has wheelbase L , longitudinal speed v at the rear axle, yaw angle ψ relative to the world x -axis, and a front steering angle δ measured from the longitudinal axis of the body. The yaw rate $\dot{\psi}$ is determined by the geometry of the model.

Two scalar control inputs act on the model: the longitudinal speed v at the rear axle, controlled through the throttle and brake pedals, and the steering angle δ at the front wheel. With these inputs, the equations of motion of the bicycle are

$$\dot{x} = v \cos \psi \quad (3.13)$$

$$\dot{y} = v \sin \psi \quad (3.14)$$

$$\dot{\psi} = \frac{v}{L} \tan \delta \quad (3.15)$$

where L is the wheelbase. For the small steering angles that occur in normal driving—the LKA function is by definition active only inside the lane, so $|\delta| < 5^\circ$ in almost every situation—the small-angle approximation $\tan \delta \approx \delta$ holds with negligible error and the yaw-rate equation simplifies to

$$\dot{\psi} = \frac{v}{L} \delta \quad (3.16)$$

In the CARLA Python API, the steering input is not the physical steering angle δ but a normalised “steer command” $\delta_c \in [-1, +1]$ that the simulator scales internally by a vehicle-specific maximum angle $\delta_{\max}$. The relationship $\delta = K_{\text{steer}} \delta_c$ with $K_{\text{steer}} \approx \delta_{\max}$ is approximately linear in the regime of interest [24]. Plugging this relationship into the

yaw-rate equation gives the relation between the actuator command and the resulting yaw rate:

$$\dot{\psi} = v \frac{K_{\text{steer}}}{L} \delta_c \quad (3.17)$$

The key observation is that $\dot{\psi}$ depends linearly on the steering command and on the speed. The factor $\frac{v}{L}$ appears because at higher speeds the same steering angle traces a larger arc per unit time. From a control-theory perspective, the steering channel of the vehicle is an *open-loop integrator* whose gain scales with speed: if the steering command is held constant, the yaw rate is constant and the heading angle grows linearly in time. This integrator is the building block from which the two plants of Section 3.4 and Section 3.5 will be constructed.

3.3 The PID Feedback Law

A Proportional–Integral–Derivative controller computes the control action $u(t)$ as a linear combination of three terms: the present error, the accumulated past error, and the rate of change of the error. Given a scalar error signal $e(t)$, the continuous-time PID law is

$$u(t) = K_p e(t) + K_i \int_0^t e(\tau) d\tau + K_d \frac{de(t)}{dt} \quad (3.18)$$

Each term plays a specific role in the LKA context. The proportional term $K_p e(t)$ produces an immediate reaction whose magnitude is proportional to the present error and is responsible for the fast component of the response. The integral term $K_i \int e(\tau) d\tau$ accumulates the past error and removes the steady-state offset that would otherwise persist when the plant has finite DC gain or when the disturbance is constant—for instance on a constantly cambered road, or under a small wheel-alignment offset. The derivative term $K_d \dot{e}(t)$ anticipates the future evolution of the error from its rate of change and adds damping when the error is approaching zero. The qualitative effect of changing each gain on the closed-loop response is summarised in Table 3.2; these dependencies are well known from the control literature [29] and are repeated here only because they directly inform the manual tuning that has been performed during the development.

Gain	Rise time	Overshoot	Settling time	Steady-state error
Increase K_p	decreases	increases	small change	decreases
Increase K_i	decreases	increases	increases	eliminates
Increase K_d	small change	decreases	decreases	no effect

Table 3.2: Qualitative effect of each PID gain on the closed-loop response of a generic plant.

3.3.1 Discrete-Time Implementation

The controller runs at the same frequency as the simulator, that is at $T_s = 0.05$ s. The continuous-time integral and derivative therefore have to be approximated. Using the backward-Euler rule for the integral and a first-order backward difference for the derivative, the PID law in Equation (3.18) becomes

$$I[k] = I[k - 1] + e[k]T_s \quad (3.19)$$

$$D[k] = \frac{e[k] - e[k - 1]}{T_s} \quad (3.20)$$

$$u[k] = K_p e[k] + K_i I[k] + K_d D[k] \quad (3.21)$$

In the actual implementation, the integral and the derivative are not stored as scalar variables but as the running sum and the first difference of a fixed-length deque of the most recent error samples. This windowed formulation provides a built-in saturation of the integral term—the deque contains at most ten samples in the present implementation—and it makes the derivative term less sensitive to high-frequency noise on the error signal.

3.3.2 Anti-Windup and Saturation

Two practical issues must be addressed in any discrete realisation of Equation (3.21). The first is *integral wind-up*: when the steering command saturates at the physical limit of the actuator, the integral term keeps growing as long as the lateral error does not change sign. Once the vehicle finally crosses the centre of the lane, the controller has to “discharge” this accumulated integral before it can produce a counter-steering action, which leads to a long undershoot. The simplest remedy, used in all three architectures of the present work, is to clamp the integral to a symmetric interval $[-I_{\max}, +I_{\max}]$ at every step.

The second issue is *output saturation* proper. The CARLA `VehicleControl.steer` field accepts values in $[-1, +1]$, with -1 corresponding to a fully left and $+1$ to a fully right steering wheel. Any control law must therefore saturate its output to this range; in the implementation, the maximum value is set to 0.8 , slightly below the physical limit, in order to leave a margin for transient overshoots—a choice that matches the default of the CARLA reference local planner [30].

A third, more subtle issue is the *rate of change* of the steering command. A PID controller is intrinsically a memoryless operator on the error sequence and can produce arbitrarily large jumps between consecutive samples, especially if the derivative term is large. Such jumps would be felt by the vehicle as a step on the steering rack and would excite high-frequency suspension dynamics. To avoid this, the steering command is rate-limited at every tick to a maximum increment of 0.1 per step, which corresponds approximately to a $50^\circ/\text{s}$ steering-wheel speed at the saturation level—well within the

capabilities of a human driver, and well below the bandwidth at which the simulator becomes numerically unstable.

3.4 Cross-Track Error Formulation

The first of the three architectures closes the loop on the *cross-track error* e_y , defined as the signed perpendicular distance between the vehicle and the closest waypoint on the reference path. The CARLA waypoint graph exposes, at every position of the world, the closest waypoint together with the unit forward vector $\hat{t} = (t_x, t_y)$ tangent to the lane. Given the vehicle position $P_v = (x_v, y_v)$ and the closest waypoint $P_w = (x_w, y_w)$, the cross-track error is the projection of the displacement $\vec{r} = P_w - P_v$ on the unit normal $\hat{n} = (-t_y, t_x)$:

$$e_y = \vec{r} \cdot \hat{n} = (x_w - x_v)(-t_y) + (y_w - y_v)t_x \quad (3.22)$$

which, after rearrangement of the signs and division by the norm of $\hat{t}$ in case the forward vector returned by the API is not exactly normalised, becomes

$$e_y = \frac{(x_w - x_v)t_y - (y_w - y_v)t_x}{\sqrt{t_x^2 + t_y^2}} \quad (3.23)$$

The geometric construction has been illustrated in Figure 3.3 and is not repeated here.

Lane-shift / cross-track geometry

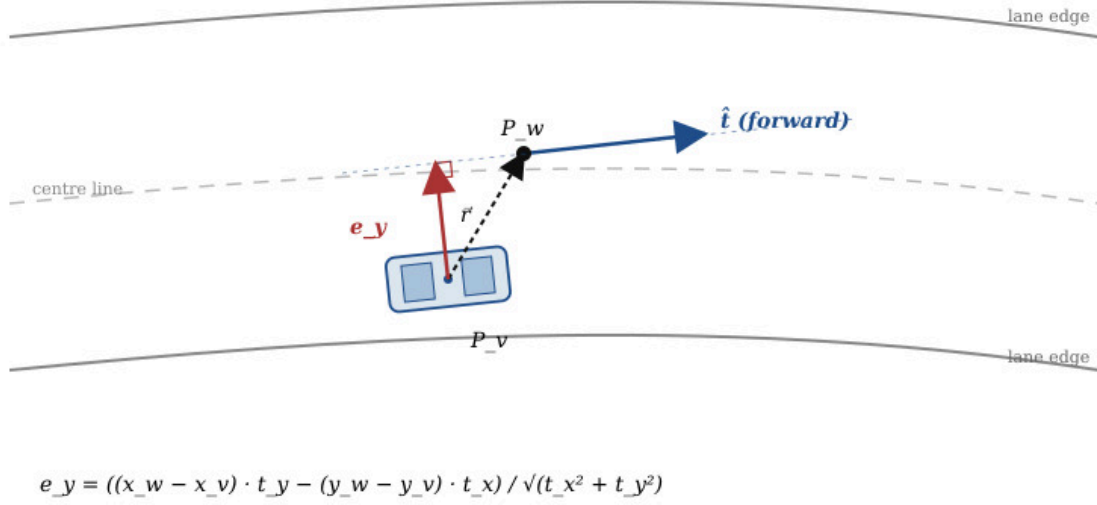


Figure 3.3: Geometric definition of the cross-track error e_y as the signed perpendicular distance between the vehicle position P_v and the line through the closest waypoint P_w in the direction of the lane forward vector $\hat{t}$. Positive values of e_y correspond to the vehicle being on the right of the lane centre with respect to the direction of travel.

3.4.1 The Cross-Track Plant

To derive the transfer function from the steering command to the cross-track error, three simple integrations have to be chained. Starting from the bicycle-model relation $\dot{\psi} = vK_{\text{steer}} \frac{\delta_c}{L}$ established in Section 3.2:

- The yaw rate $\dot{\psi}$ integrated once gives the heading deviation ψ from the path tangent.
- The heading deviation ψ multiplied by the vehicle speed v gives the lateral velocity $\dot{y}$ relative to the path (small-angle approximation).
- The lateral velocity $\dot{y}$ integrated once gives the cross-track distance e_y .

In the Laplace domain, the chain of operations is

$$\frac{E_y(s)}{\Delta_c(s)} = \frac{vK_{\text{steer}}}{L} \cdot \frac{1}{s} \cdot v \cdot \frac{1}{s} = \frac{v^2 K_{\text{steer}}}{Ls^2} \quad (3.24)$$

The plant from the steering command to the cross-track error is therefore a *double integrator*, with a DC gain that scales with v^2 . This is the central observation that drives every other property of the cross-track formulation. A double integrator is marginally

stable on its own—it has two poles at the origin—and any feedback law that uses only the proportional and integral terms produces a closed-loop system with poles either on the imaginary axis or in the right half-plane. The derivative term is therefore *necessary* for stability, not optional. Furthermore, the v^2 scaling of the DC gain means that, in order to keep the closed-loop bandwidth constant as the vehicle accelerates, all three PID gains have to be re-scaled by $\frac{1}{v^2}$. This is a steep dependence that will be revisited in Section 3.6.

3.4.2 Pole Placement

A natural choice for the closed-loop denominator of a PID compensating a double integrator is a triple real pole at $s = -\omega_n$, which gives a critically damped behaviour with bandwidth ω_n . The closed-loop characteristic polynomial of $K_p + \frac{K_i}{s} + K_d s$ acting on $\frac{K_{\text{lat}}}{s^2}$ with $K_{\text{lat}} = v^2 \frac{K_{\text{steer}}}{L}$ is

$$s^3 + (K_{\text{lat}} K_d) s^2 + (K_{\text{lat}} K_p) s + K_{\text{lat}} K_i \quad (3.25)$$

Matching with $(s + \omega_n)^3 = s^3 + 3\omega_n s^2 + 3\omega_n^2 s + \omega_n^3$ yields the analytical gains

$$K_p = \frac{3\omega_n^2}{K_{\text{lat}}}, \quad K_i = \frac{\omega_n^3}{K_{\text{lat}}}, \quad K_d = \frac{3\omega_n}{K_{\text{lat}}} \quad (3.26)$$

These expressions tell, at a glance, how each gain depends on the vehicle speed: K_d scales as $\frac{1}{v^2}$ through K_{lat} , K_p does the same, and K_i —the most “expensive” gain in terms of stability margin—decreases as $v^{\{-2\}}$ as well. The tuning script `pid_tuning.py` implements exactly Equation (3.26) and produces, given a measurement of K_{steer} from the system identification of Section 3.6, the numerical gains that have been used in the simulator.

3.5 Heading-Error Formulation

The second architecture replaces the cross-track distance with a different scalar error signal: the angle between the vehicle’s forward direction and the line connecting the vehicle to a *look-ahead point* placed on the reference path at a speed-dependent distance L_d ahead of the vehicle. This formulation is closely related to the Pure Pursuit [27] and Stanley [26] geometric trackers, with the difference that here the steering command is computed by a feedback PID rather than by a fixed geometric formula. The geometric construction is illustrated in Figure 3.4.

Heading-error / look-ahead geometry

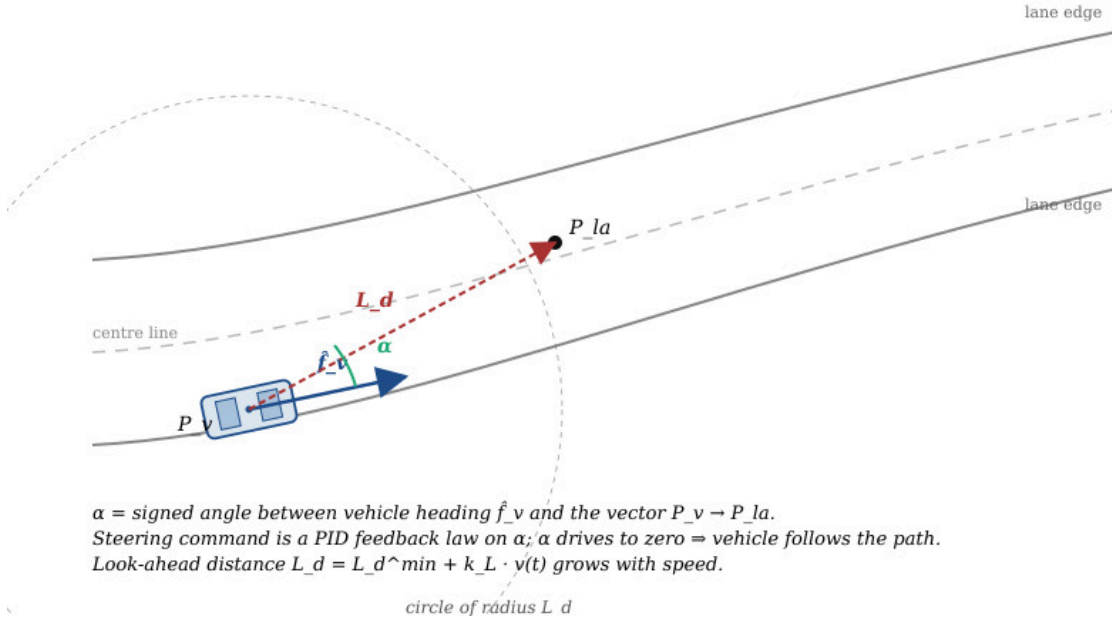


Figure 3.4: Heading-error / look-ahead geometry. The look-ahead point P_{la} is the closest waypoint on the reference path at distance at least L_d ahead of the vehicle; the heading error α is the signed angle between the vehicle forward unit vector $\hat{f}_v$ and the line $P_v \rightarrow P_{la}$. The look-ahead distance is scheduled as $L_d = L_d^{\min} + k_L v(t)$.

Formally, given the vehicle position $P_v = (x_v, y_v)$, its forward unit vector $\hat{f}_v = (f_x, f_y)$, and the look-ahead point $P_{la} = (x_l, y_l)$, the heading error α is computed from the dot product and the cross product of $\hat{f}_v$ with the displacement $\vec{d} = P_{la} - P_v$:

$$\alpha = \text{atan2}(\hat{f}_v \times \vec{d}, \hat{f}_v \cdot \vec{d}) \quad (3.27)$$

The two-argument arctangent returns a signed angle in $(-\pi, +\pi]$ and naturally handles all four quadrants. By convention—the same one used in the CARLA `agents.navigation.controller.PIDLateralController` and matched in the implementation—a positive α corresponds to a target on the right of the vehicle, which calls for a positive (right) steering command.

3.5.1 The Heading-Error Plant

Compared with the cross-track formulation, the heading-error plant has *one fewer integration*. Starting again from $\dot{\psi} = v K_{\text{steer}} \frac{\delta_c}{L}$, only one step is needed to obtain the heading deviation α relative to the look-ahead point. The look-ahead point is, by construction, ahead of the vehicle on the path; if the path is locally straight, the line to the look-ahead point is parallel to the path tangent, and the heading error reduces to the angle between the vehicle's forward vector and the path tangent. In the Laplace domain,

$$\frac{A(s)}{\Delta_{c(s)}} = \frac{vK_{\text{steer}}}{L} \cdot \frac{1}{s} = \frac{vK_{\text{steer}}}{Ls} \quad (3.28)$$

which is a *single integrator* with a DC gain that scales linearly with v , not with v^2 .

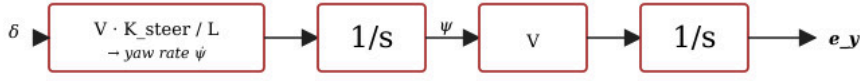
The structural difference between Equation (3.24) and Equation (3.28) is the single most important property of the present chapter, because it has three direct consequences:

- A PI controller is sufficient for closed-loop stability. The derivative term is no longer necessary—the plant has only one free integrator—and is included only if the error signal is noisy enough to require derivative filtering.
- The gains scale as $\frac{1}{v}$ instead of $\frac{1}{v^2}$. Gain scheduling across the operating speed range is therefore much gentler.
- The closed-loop bandwidth that the controller can achieve at fixed gains is higher, because each additional integrator in the plant adds 90° of phase lag and reduces the available phase margin.

The plant comparison is summarised in Figure 3.5, which puts the two transfer functions side by side and makes the role of the look-ahead point explicit: by closing the loop on the angle to a point on the path rather than on the perpendicular distance to it, the controller “consumes” one of the two integrators that the cross-track formulation has to handle.

Plant model comparison: cross-track vs. heading-error formulation

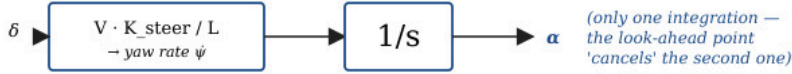
A. Cross-track error e_y formulation



Combined: $G_y(s) = (V^2 \cdot K_{steer} / L) / s^2 \rightarrow$ **double integrator**

- Plant gain scales with V^2 — gains must scale $1/V^2$ to maintain bandwidth.
- PID needed: D-term essential for stability (two free integrators).
- Pole placement: $(s + \omega_n)^3 = K_p = 3\omega_n^2/K_{lat}$, $K_i = \omega_n^3/K_{lat}$, $K_d = 3\omega_n/K_{lat}$.

B. Heading-error α formulation



Combined: $G_\alpha(s) = (V \cdot K_{steer} / L) / s \rightarrow$ **single integrator**

- Plant gain scales linearly with V — gains scale $1/V$ (gentler).
- PI is sufficient for stability; D-term optional, only for noise filtering.
- Pole placement: $(s + \omega_n)^2 = K_p = 2\zeta\omega_n/K_{lat}$, $K_i = \omega_n^2/K_{lat}$.

The choice of error signal (e_y or α) determines the plant order; the plant order determines what controller is needed.

Figure 3.5: Block-diagram comparison of the cross-track and heading-error plants. The cross-track formulation contains two integrators—yaw rate to heading, and lateral velocity to cross-track distance—while the heading-error formulation contains only one. The reduction in plant order is the reason why a PI controller is sufficient for the heading-error case, while a full PID is required for the cross-track case.

3.5.2 Pole Placement

With the plant given by Equation (3.28) and a PI controller $C(s) = K_p + \frac{K_i}{s}$, the closed-loop characteristic polynomial reads

$$s^2 + K_{lat}K_p s + K_{lat}K_i, \quad K_{lat} = v \frac{K_{steer}}{L} \quad (3.29)$$

Matching with $s^2 + 2\zeta\omega_n s + \omega_n^2$ and choosing the critical damping $\zeta = 1$ yields

$$K_p = \frac{2\omega_n}{K_{lat}}, \quad K_i = \frac{\omega_n^2}{K_{lat}} \quad (3.30)$$

Both gains scale as $\frac{1}{v}$ through K_{lat} . The asymptotic comparison with the cross-track formulation is best read graphically: Figure 3.6 shows, in the bottom-right panel, the proportional gain K_p as a function of vehicle speed for the two formulations, normalised so that the value at 30 km/h is unity. The cross-track curve falls by an order of magnitude when the speed grows from 30 to 120 km/h, while the heading-error curve only falls by a factor of four. The practical consequence is that the same heading-error PI controller can be safely operated, with at most a mild gain re-scheduling, across the entire highway-

speed range; the cross-track PID, in contrast, is much more sensitive to speed mismatches and would oscillate or diverge if used outside the range it has been tuned for.

3.5.3 Look-Ahead Distance Scheduling

The look-ahead point introduces a single design parameter, the look-ahead distance L_d . A short L_d produces a controller that reacts strongly to local geometry and that is well-behaved on tight turns but easily destabilised at high speed; a long L_d produces a smooth controller that filters out high-frequency disturbances on the reference path but cuts corners on tight turns. The compromise that is most often adopted in the literature on Pure Pursuit—and that has been retained in the present implementation—is to schedule L_d linearly with the speed of the vehicle:

$$L_{d(v)} = L_d^{\min} + k_L v \quad (3.31)$$

with a minimum $L_d^{\min} = 4$ m that protects against pathological behaviour at standstill, and a slope $k_L = 0.6$ s that produces $L_d \approx 9$ m at the nominal cruise speed of 30 km/h. The same kind of linear scheduling is used by the CARLA reference local planner [30] for its waypoint-popping criterion, with comparable numerical values.

3.5.4 Heading-Error Filtering

A practical consideration that is specific to the heading-error formulation is the noise on the error signal. The look-ahead point is selected from a discrete sampling of the reference path, so the heading error is intrinsically a piecewise-constant signal that exhibits small jumps every time the look-ahead point advances by one waypoint. These jumps are amplified by the derivative term and would produce visible jitter on the steering command. In the implementation, the heading error is therefore low-pass filtered with a first-order infinite impulse response filter of pole α_H :

$$\tilde{\alpha}[k] = \alpha_H \alpha[k] + (1 - \alpha_H) \tilde{\alpha}[k - 1] \quad (3.32)$$

with $\alpha_H = 0.4$, which corresponds to a cut-off frequency of approximately 2 Hz at the simulator rate of 20 Hz. This is well below the Nyquist frequency of the controller and well above the bandwidth of the closed loop, so it removes the per-sample jitter without affecting the dynamic response of the system.

3.6 System Identification and Analytical Tuning

The pole-placement formulas Equation (3.26) and Equation (3.30) express the PID gains in terms of two physical parameters: the wheelbase L , which is read from the vehicle blueprint through the CARLA Python API, and the steering gain K_{steer} , which depends on the specific vehicle model and is not exposed by the API. The latter has to be measured experimentally. In addition, the longitudinal channel of the LKA function—the

speed regulator that maintains the cruise speed—has its own parameters that have to be identified.

The identification procedure has been implemented in the standalone script `carla_sysid.py`; it consists of two open-loop step experiments that are recorded with the simulator in synchronous mode at $T_s = 0.05$ s.

3.6.1 Longitudinal Step Experiment

The first experiment fits the parameters of a first-order longitudinal model. With the vehicle initially at rest on a long straight road, the throttle is stepped from zero to a fixed value $u_{\text{step}} = 0.5$ and held for ten seconds. The resulting speed trajectory $v(t)$ is recorded and fitted, by nonlinear least squares, to the first-order step response

$$v(t) = Ku_{\text{step}} \left(1 - e^{\{-\frac{t}{\tau}\}}\right) \quad (3.33)$$

where K is the DC gain (in km/h per unit of throttle) and τ is the time constant of the longitudinal dynamics. The model is sufficient because at the moderate accelerations encountered during the experiments, the dominant non-linearity of the powertrain—the throttle-to-torque map—is approximately linear, and the slower aerodynamic drag becomes important only above 80 km/h.

3.6.2 Lateral Step Experiment

The second experiment identifies the steering gain K_{steer} . The vehicle is settled at a constant speed $V = 30$ km/h with a simple proportional speed regulator, and a small steering step $\delta_c = 0.05$ is applied for 2.5 s. The yaw rate of the vehicle is recorded directly from the `get_angular_velocity` getter of the CARLA Python API. The steady-state yaw rate $\dot{\psi}_{\text{ss}}$ obtained by averaging the last 25% of the trace is then matched against the bicycle-model prediction $\dot{\psi}_{\text{ss}} = \left(\frac{V}{L}\right)K_{\text{steer}}\delta_c$, giving

$$K_{\text{steer}} = \frac{\dot{\psi}_{\text{ss}}L}{V\delta_c} \quad (3.34)$$

The duration of the lateral step is kept deliberately short, because once the vehicle starts to drift laterally the cross-track error grows quickly and the small-angle assumptions of the bicycle model are no longer satisfied.

3.6.3 Analytical Plot Comparison

With L and K_{steer} identified, the analytical gains of Equation (3.26) and Equation (3.30) can be evaluated and the resulting closed-loop transfer functions can be analysed without running the simulator. Figure 3.6 shows the four canonical control plots that the tuning script `pid_tuning.py` produces from the identified plant parameters: the open-loop magnitude and phase of the loop transfer $L(s) = C(s)G(s)$ for the two formulations (top

row), the closed-loop step response of the reference-to-output transfer $T(s)$ (bottom-left), and the speed dependence of the proportional gain K_p (bottom-right).

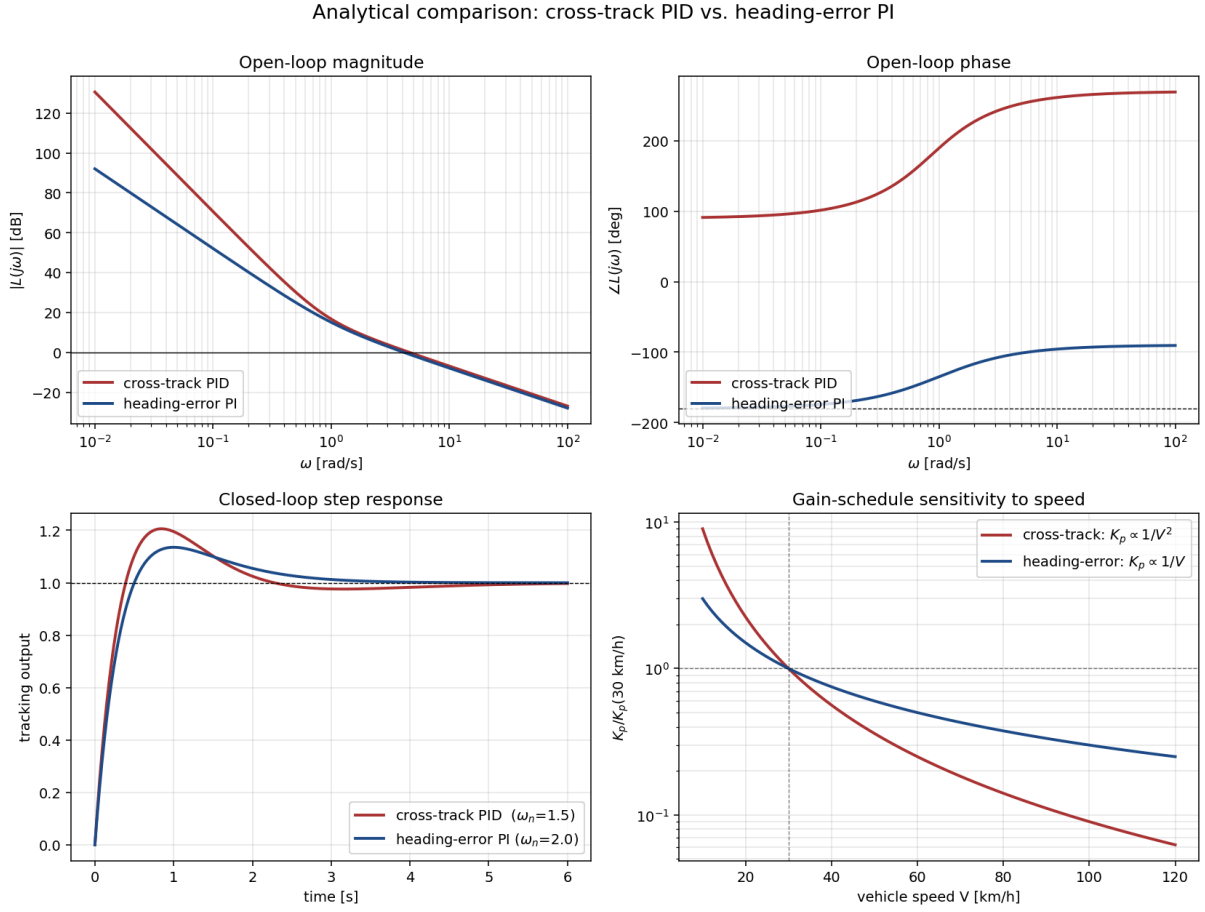


Figure 3.6: Analytical comparison of the cross-track PID and heading-error PI controllers, computed symbolically from the identified plant parameters. *Top*: open-loop magnitude and phase. *Bottom-left*: closed-loop step response. *Bottom-right*: proportional gain as a function of vehicle speed, normalised at 30 km/h. The gentler $\frac{1}{v}$ scaling of the heading-error gain, compared with the $\frac{1}{v^2}$ scaling of the cross-track gain, is the main reason why the heading-error formulation is preferred in the operational range of an LKA function.

Several observations can be made from these plots. The two open-loop magnitudes coincide above $\omega \approx 4$ rad/s—at high frequency, both loops are dominated by the fastest pole of the controller—while at low frequency the cross-track loop has a steeper slope that reflects the additional integrator. The phase of the cross-track loop reaches -180° around the same frequency at which the heading-error loop is still well above -90° , which translates into a smaller phase margin and a more oscillatory step response. Both controllers track a unit step, but the cross-track design exhibits a larger overshoot and a longer settling time, even though its bandwidth ω_n has been deliberately set lower than that of the heading-error controller to compensate for the additional plant order.

The bottom-right panel quantifies the gain-scheduling argument that has been made in Section 3.5. At $V = 120$ km/h, the proportional gain of the cross-track controller is more than ten times smaller than its value at 30 km/h, while the same gain of the heading-error controller is only four times smaller. In a fixed-gain implementation, this means that a cross-track PID tuned at 30 km/h is unsafe to use even at 50 km/h, while a heading-error PI tuned at the same speed is acceptable for the entire range 20–60 km/h that covers urban driving and most of the operational design domain of an LKA function.

3.7 Vision-Based Lane Reference

The third architecture removes the dependence on the pre-recorded `.csv` reference path by reconstructing the centre line of the lane at every tick from the RGB camera feed. The architecture is identical to the heading-error PI controller of Section 3.5 from the controller block downward; the only modification is the way in which the look-ahead point is generated. A high-level view of the vision-based pipeline is given on the bottom row of Figure 3.1.

3.7.1 The Lane-Detection Network

Lane detection in road scenes has been a benchmark task in computer vision for over a decade, and the literature offers a variety of architectures with different trade-offs between accuracy, speed, and complexity. The taxonomy adopted in the recent comparative study of Lin et al. [31] covers semantic-segmentation networks (SCNN [32], RESA [33]), row-based classifiers (UFLD [34]), anchor-based detectors (LaneATT [35], ADNet [36]), parametric-curve regressors (BezierLaneNet [37]) and the recent CLRNet [38] family that combines an anchor-based proposal mechanism with row-wise refinement. The implementation supports any model from this list through a pluggable registry stored in the `detection_models` dictionary; for the experiments reported in this thesis, the CLRNet model trained on the TuSimple dataset has been used, because it offers the best trade-off between latency and lane-fitting accuracy in the comparative study cited above.

The detection network operates on a 1280×720 frame extracted from the camera sensor and produces a list of lane lines, each represented as a sequence of pixel coordinates (u_i, v_i) . In the present formulation, the reference for the controller is not a single lane line but the *centre line* of the ego-lane, which has to be reconstructed from the detected boundaries. The reconstruction is performed by the helper function `get_centerline`: only the lane lines that reach the bottom 30% of the image are kept—so that adjacent lanes and far-away lines are filtered out—and the closest left and right boundaries to the image centre are paired up. Their interpolated mid-points at a regular sampling along the image v -axis form the centre line, which is returned ordered from the closest sample to the furthest.

3.7.2 Inverse Perspective Mapping

The centre line returned by the lane-detection network is expressed in pixel coordinates and cannot be used directly by the controller, which works in the body frame of the vehicle. The conversion from pixels to a body-frame position is performed by the standard *inverse perspective mapping* (IPM) projection, sketched in Figure 3.7.

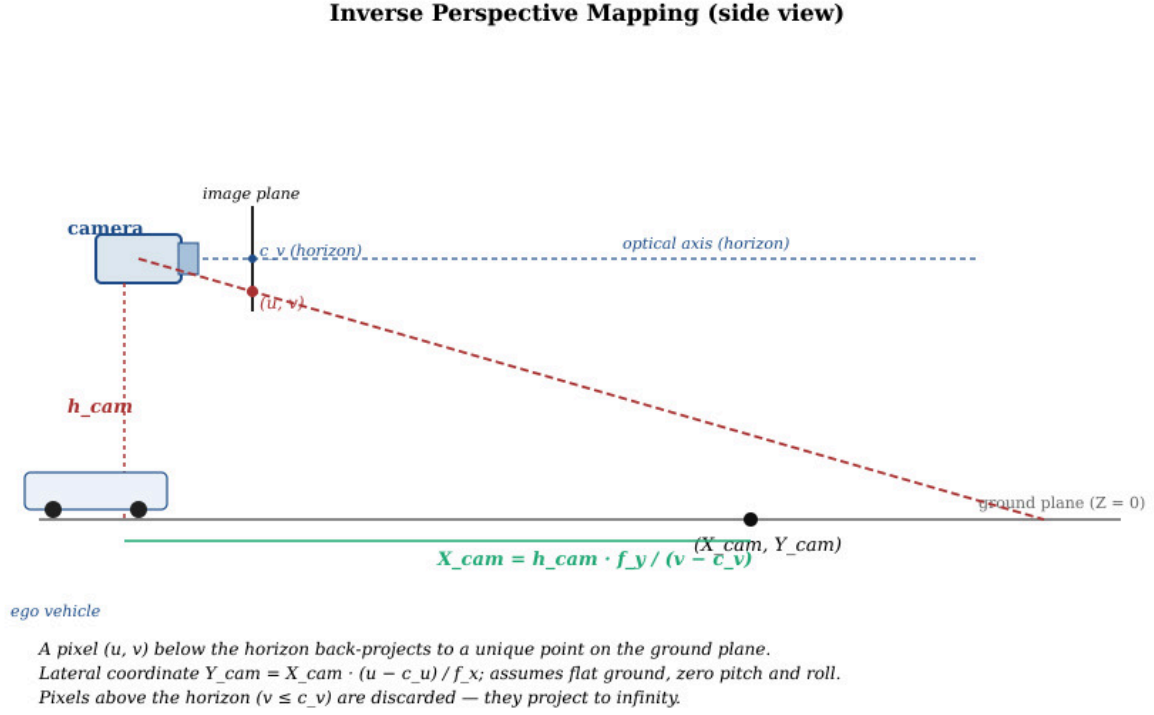


Figure 3.7: Inverse perspective mapping (side view). A pixel below the horizon back-projects to a unique point on the ground plane, given the camera height h_{cam} and the camera intrinsics (f_x, f_y, c_u, c_v) . Pixels above the horizon $v \leq c_v$ project to infinity and are discarded.

Under the assumptions of a pinhole camera with zero pitch, zero roll, zero yaw, and a flat ground plane, every pixel below the horizon corresponds to a unique point on the ground plane. With the camera mounted at height h_{cam} above the road, with focal lengths $f_x = f_y = \frac{w}{2 \tan(\frac{FOV}{2})}$ derived from the image width w and the field of view, and with the principal point at $(c_u, c_v) = (\frac{w}{2}, \frac{h}{2})$, the mapping reads

$$X_{cam} = \frac{h_{cam} f_y}{v - c_v}, \quad Y_{cam} = X_{cam} \frac{u - c_u}{f_x} \quad (3.35)$$

where X_{cam} is the longitudinal distance ahead of the camera, in metres, and Y_{cam} is the lateral distance to the right of the camera. The projection is well defined only for pixels below the horizon, that is for $v > c_v$; pixels above the horizon are discarded. A maximum range $X_{cam} \leq 40$ m is also enforced, because at larger distances the small angular resolution of the pixel produces unstable lateral estimates. The constant offset of

the camera with respect to the rear axle of the vehicle is taken into account by adding $0.8b_x$ to X_{cam} , where b_x is the half-extent of the bounding box of the vehicle.

The output of the IPM block is therefore a sampling of the centre line of the lane in the body frame of the vehicle, in metres, ordered from the closest sample to the furthest. This is exactly the input that the heading-error formulation of Section 3.5 expects, with the only difference that the look-ahead point is now selected on a perception-based reference rather than on a pre-recorded one.

3.7.3 Polynomial Fitting and Look-Ahead Selection

The samples of the centre line returned by the IPM block are noisy and unevenly spaced. To produce a stable look-ahead point, a low-degree polynomial $y = a_1x + a_0$ is fitted by least squares to the samples that fall in the longitudinal interval $[0.5, 20]$ m, and the look-ahead lateral coordinate is evaluated as $y_{\text{la}} = a_1L_d + a_0$. A first-degree polynomial—a straight line—has been adopted because the operational scenario consists of mostly straight roads and gentle curves at moderate speed; for higher speeds and tighter curves, a second-degree polynomial would be necessary, with a corresponding increase in sensitivity to outliers. The heading error is then computed as

$$\alpha = \text{atan2}(y_{\text{la}}, L_d) \quad (3.36)$$

which is the body-frame counterpart of the world-frame definition given in Section 3.5. The same PI controller, with the same low-pass filter on α , then produces the steering command.

3.7.4 Junction Handling

A specific issue that arises with the vision-based formulation, but not with the path-based formulations, is the disappearance of the lane markings inside an intersection. CARLA, like real-world road infrastructure, does not paint lane markings inside junctions, so the lane-detection network either returns nothing or returns lines that belong to the entry and exit lanes of different branches. To prevent the controller from acting on this unreliable input, a junction-detection mechanism has been added on top of the perception pipeline. At every tick, the closest waypoint is queried with `is_junction`; if either this flag is set, or the vehicle is inside one of three pre-flagged buffer regions where the lane markings are known to disappear before `is_junction` flips, the LKA function is temporarily disabled and the control is handed off to the CARLA Traffic Manager autopilot until the vehicle exits the junction. A reset of the integral state of the PI controller and of the look-ahead-distance filter is performed at every re-engagement, so that no transient is propagated from one intersection to the next.

3.8 Method Comparison

The three architectures presented in this chapter solve the same lane-keeping problem with progressively richer information and progressively more sophisticated control structure. From a purely control-theoretic standpoint, they can be summarised as in Table 3.3.

Property	Cross-track PID	Heading-error PI	Vision-based PI
Reference signal	recorded path	recorded path	reconstructed centre line
Error signal	perpendicular distance e_y	look-ahead angle α	look-ahead angle α
Plant order	2 (double integrator)	1 (single integrator)	1 (single integrator)
Required terms	$P + I + D$	$P + I$ (D optional)	$P + I$ (D optional)
Plant gain scaling	$\propto v^2$	$\propto v$	$\propto v$
Gain scheduling	$\propto 1/v^2$	$\propto 1/v$	$\propto 1/v$
Phase margin	smaller	larger	larger
Robustness to noise	moderate	good	depends on detector
Map dependence	yes (pre-recorded)	yes (pre-recorded)	no
Generalisation	restricted to recorded route	restricted to recorded route	unrestricted within ODD

Table 3.3: Side-by-side comparison of the three lateral-control architectures along the dimensions that are most relevant for an LKA function.

The progression from one architecture to the next is informed by a clear control-theoretic rationale rather than by an incremental engineering choice. The move from the cross-track to the heading-error formulation is motivated by the reduction in plant order: by closing the loop on the angle to a look-ahead point rather than on the perpendicular distance to the path, one of the two integrators of the cross-track plant is removed, the derivative term becomes optional, the gain schedule becomes gentler, and the phase margin becomes larger at any fixed bandwidth. The move from the path-based heading-error formulation to the vision-based one is motivated by the removal of the dependence on a pre-recorded reference: the same controller can be used on any road that the lane-detection network has been trained for, at the cost of an additional perception layer that has its own failure modes—most notably inside intersections, which has motivated the introduction of an explicit junction-handling logic.

The numerical evaluation of the three architectures on a common test scenario is the subject of the next chapter, which will quantify the qualitative arguments developed here in terms of cross-track error, lateral RMSE, and steering-command smoothness.

Bibliography

- [1] “Waymo - Self-Driving Cars - Autonomous vehicles - Ride-Hail.” [Online]. Available: <https://waymo.com/>
- [2] A. Kumar and S. Palaniswamy, “Steering Angle Estimation for Self-driving Car Using Deep Learning,” 2020, pp. 196–207. doi: 10.1007/978-981-15-4301-2_17.
- [3] A. R. Plummer, “Model-in-the-Loop testing,” *Proceedings of the Institution of Mechanical Engineers Part I Journal of Systems and Control Engineering*, vol. 220, no. 3, pp. 183–199, May 2006, doi: 10.1243/09596518jsce207.
- [4] S. Demers, P. Gopalakrishnan, and L. Kant, “A Generic Solution to Software-in-the-Loop,” in *MILCOM 2007 - IEEE Military Communications Conference*, 2007, pp. 1–6. doi: 10.1109/MILCOM.2007.4455268.
- [5] H. K. Fathy, Z. S. Filipi, J. Hagena, and J. L. Stein, “Review of hardware-in-the-loop simulation and its prospects in the automotive area,” *Proceedings of SPIE, the International Society for Optical Engineering/Proceedings of SPIE*, May 2006, doi: 10.1117/12.667794.
- [6] M. Abboush, C. Knieke, and A. Rausch, “Hardware-in-the-Loop-Based Real-Time Fault Injection Framework for Dynamic Behavior Analysis of Automotive Software Systems,” *Sensors*, vol. 22, p. 24, 2022, doi: 10.3390/s22041360.
- [7] Admin, “COMPACT Simulator.” [Online]. Available: http://tenergysoft.com/?page_id=13571&utm_source=AdsDzr.ru
- [8] A. Dosovitskiy, G. Ros, F. Codevilla, A. Lopez, and V. Koltun, “CARLA: an open urban driving simulator.” [Online]. Available: <http://arxiv.org/abs/1711.03938>
- [9] S. Shah, D. Dey, C. Lovett, and A. Kapoor, “AiRSIM: High-Fidelity visual and Physical simulation for autonomous vehicles.” [Online]. Available: <http://arxiv.org/abs/1705.05065>
- [10] C. Gulino *et al.*, “Waymax: An Accelerated, Data-Driven Simulator for Large-Scale Autonomous Driving Research.” [Online]. Available: <https://arxiv.org/abs/2310.08710>
- [11] G. Rong *et al.*, “LGSVL Simulator: A High Fidelity Simulator for Autonomous Driving.” [Online]. Available: <https://arxiv.org/abs/2005.03778>
- [12] M. Müller, “Sim4CV.” [Online]. Available: <https://sim4cv.org/>
- [13] C. Team, “CARLA.” [Online]. Available: <https://carla.org/>
- [14] E. Coelingh, A. Eidehall, and M. Bengtsson, “Collision Warning with Full Auto Brake and Pedestrian Detection - a practical example of Automatic Emergency Braking,” in *13th International IEEE Conference on Intelligent Transportation Systems*, Sept. 2010, pp. 155–160. doi: 10.1109/ITSC.2010.5625077.

- [15] M.-Y. Fu and Y.-S. Huang, “A survey of traffic sign recognition,” *International Conference on Wavelet Analysis and Pattern Recognition*, pp. 119–124, 2010, doi: 10.1109/icwapr.2010.5576425.
- [16] L. Küpper and J. Schug, “Active Night Vision systems,” *SAE technical papers on CD-ROM/SAE technical paper series*, 2002, doi: 10.4271/2002-01-0013.
- [17] W. contributors, “Fatigue detection software.” [Online]. Available: https://en.wikipedia.org/wiki/Fatigue_detection_software
- [18] G. Liu, M. Zhou, L. Wang, H. Wang, and X. Guo, “A blind spot detection and warning system based on millimeter wave radar for driver assistance,” *Optik*, vol. 135, pp. 353–365, 2017, doi: 10.1016/j.ijleo.2017.01.058.
- [19] Y. Song and C. Liao, “Analysis and review of state-of-the-art automatic parking assist system,” in *2016 IEEE International Conference on Vehicular Electronics and Safety (ICVES)*, 2016, pp. 1–6. doi: 10.1109/ICVES.2016.7548171.
- [20] M. C. D. What, “Adaptive cruise control - My car does what.” [Online]. Available: <https://mycardoeswhat.org/deeper-learning/adaptive-cruise-control/>
- [21] W. Son, Y. Ha, T. Oh, S. Woo, S. Cho, and J. Yoo, “PG-Based Vehicle-In-the-Loop Simulation for System Development and Consistency Validation,” *Electronics*, vol. 11, no. 24, 2022, doi: 10.3390/electronics11244073.
- [22] J. Cheng, Z. Wang, X. Zhao, Z. Xu, M. Ding, and K. Takeda, “A Survey on Testbench-Based Vehicle-in-the-Loop Simulation Testing for Autonomous Vehicles: Architecture, Principle, and Equipment,” *Advanced Intelligent Systems*, vol. 6, no. 6, p. 2300778, 2024, doi: <https://doi.org/10.1002/aisy.202300778>.
- [23] X. Hu, J. Zeng, S. Li, W. Liao, X. Ding, and Y. Ren, “Vehicle-in-loop simulation test research for autonomous vehicle,” in *Eighth International Conference on Electro-mechanical Control Technology and Transportation (ICECTT 2023)*, S. Easa and W. Wei, Eds., SPIE, 2023, p. 127900T. doi: 10.1117/12.2690076.
- [24] Y. Kebbati, N. Ait-Oufroukh, D. Ichalal, and V. Vigneron, “Lateral control for autonomous wheeled vehicles: A technical review,” *Asian Journal of Control*, vol. 25, no. 4, pp. 2539–2563, 2023, doi: <https://doi.org/10.1002/asjc.2980>.
- [25] A. Artuñedo, M. Moreno-Gonzalez, and J. Villagra, “Lateral control for autonomous vehicles: A comparative evaluation,” *Annual Reviews in Control*, vol. 57, p. 100910, 2024, doi: <https://doi.org/10.1016/j.arcontrol.2023.100910>.
- [26] S. Dominguez, A. Ali, G. Garcia, and P. Martinet, “Comparison of lateral controllers for autonomous vehicle: Experimental results,” in *2016 IEEE 19th International Conference on Intelligent Transportation Systems (ITSC)*, 2016, pp. 1418–1423. doi: 10.1109/ITSC.2016.7795743.
- [27] J. M. Snider and others, “Automatic steering methods for autonomous automobile path tracking,” *Robotics Institute, Pittsburgh, PA, Tech. Rep. CMU-RITR-09-08*, 2009.

- [28] W. contributors, “Unreal Engine 4.” [Online]. Available: https://en.wikipedia.org/wiki/Unreal_Engine_4
- [29] K. J. Åström and T. Hägglund, *Advanced PID Control*. Research Triangle Park, NC: ISA - The Instrumentation, Systems,, Automation Society, 2006.
- [30] CARLA Team, “CARLA Python API: agents.navigation.local_planner.” 2024.
- [31] Y.-W. Lin and C.-H. Tsai, “A Comprehensive Review of Deep-Learning-Based Lane Detection Methods for Autonomous Driving,” *IEEE Access*, vol. 12, pp. 12345–12367, 2024.
- [32] X. Pan, J. Shi, P. Luo, X. Wang, and X. Tang, “Spatial As Deep: Spatial CNN for Traffic Scene Understanding,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, 2018.
- [33] T. Zheng *et al.*, “RESA: Recurrent Feature-Shift Aggregator for Lane Detection,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, 2021, pp. 3547–3554.
- [34] Z. Qin, H. Wang, and X. Li, “Ultra Fast Structure-Aware Deep Lane Detection,” in *European Conference on Computer Vision (ECCV)*, 2020, pp. 276–291.
- [35] L. Tabelini, R. Berriel, T. M. Paixão, C. Badue, A. F. De Souza, and T. Oliveira-Santos, “Keep Your Eyes on the Lane: Real-time Attention-guided Lane Detection,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021, pp. 294–302.
- [36] L. Xiao, X. Li, S. Yang, and W. Yang, “ADNet: Lane Shape Prediction via Anchor Decomposition,” in *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023, pp. 6404–6413.
- [37] Z. Feng, S. Guo, X. Tan, K. Xu, M. Wang, and L. Ma, “Rethinking Efficient Lane Detection via Curve Modeling,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022, pp. 17062–17070.
- [38] T. Zheng *et al.*, “CLRNet: Cross Layer Refinement Network for Lane Detection,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022, pp. 898–907.

List of Figures

Figure 1.1 The historical development of autonomous driving [2].	4
Figure 1.2 MIL, SIL , PIL, HIL and VIL tests in V-cycle development process [6].	5
Figure 1.3 Driving Test Simulator [7]	6
Figure 1.4 Comparing Waymo Simulator graphics and the real world.	7
Figure 1.5 SVL Simulator Screen	8
Figure 1.6 Sim4CV: A Photo-Realistic Simulator for Computer Vision Applications	8
Figure 1.7 Carla Simulator Screen	9
Figure 1.8 Automatic Emergency Braking system.	10
Figure 1.9 Traffic Sign Recognition System	10
Figure 1.10 Night Vision	11
Figure 1.11 Blind Spot Warning System	11
Figure 1.12 Parking Asistant	11
Figure 1.13 Automatic Headlight Control	12
Figure 1.14 Lane Keeping Asistant	12
Figure 1.15 Adaptive Cruise Control	12
Figure 2.1 Single Track Model	17
Figure 2.2 Vehicle Reference System	18
Figure 2.3 Friction Coefficents in different conditions	19
Figure 3.1 Block-diagram comparison of the three lateral-control architectures investigated in this thesis. The first one closes the loop on the cross-track distance computed against a recorded path; the second one closes the loop on a heading angle derived from a speed-dependent look-ahead point; the third one replaces the recorded path with a perception pipeline that reconstructs the centre line of the lane from the camera image. All three architectures share the same actuator, the same vehicle dynamics, and the same PID structure.	25
Figure 3.2 Kinematic bicycle model. The vehicle has wheelbase L , longitudinal speed v at the rear axle, yaw angle ψ relative to the world x -axis, and a front steering angle δ measured from the longitudinal axis of the body. The yaw rate $\dot{\psi}$ is determined by the geometry of the model.	26
Figure 3.3 Geometric definition of the cross-track error e_y as the signed perpendicular distance between the vehicle position P_v and the line through the closest waypoint P_w	

in the direction of the lane forward vector $\hat{t}$. Positive values of e_y correspond to the vehicle being on the right of the lane centre with respect to the direction of travel. . .	30
Figure 3.4 Heading-error / look-ahead geometry. The look-ahead point P_{la} is the closest waypoint on the reference path at distance at least L_d ahead of the vehicle; the heading error α is the signed angle between the vehicle forward unit vector $\hat{f}_v$ and the line $P_v \rightarrow P_{\text{la}}$. The look-ahead distance is scheduled as $L_d = L_d^{\text{min}} + k_L v(t)$	32
Figure 3.5 Block-diagram comparison of the cross-track and heading-error plants. The cross-track formulation contains two integrators—yaw rate to heading, and lateral velocity to cross-track distance—while the heading-error formulation contains only one. The reduction in plant order is the reason why a PI controller is sufficient for the heading-error case, while a full PID is required for the cross-track case.	34
Figure 3.6 Analytical comparison of the cross-track PID and heading-error PI controllers, computed symbolically from the identified plant parameters. <i>Top</i> : open-loop magnitude and phase. <i>Bottom-left</i> : closed-loop step response. <i>Bottom-right</i> : proportional gain as a function of vehicle speed, normalised at 30 km/h. The gentler $\frac{1}{v}$ scaling of the heading-error gain, compared with the $\frac{1}{v^2}$ scaling of the cross-track gain, is the main reason why the heading-error formulation is preferred in the operational range of an LKA function.	37
Figure 3.7 Inverse perspective mapping (side view). A pixel below the horizon back-projects to a unique point on the ground plane, given the camera height h_{cam} and the camera intrinsics (f_x, f_y, c_u, c_v) . Pixels above the horizon $v \leq c_v$ project to infinity and are discarded.	39

List of Tables

Table 1.1 Comparative Feature Scoring of Simulators	9
Table 3.2 Qualitative effect of each PID gain on the closed-loop response of a generic plant.	27
Table 3.3 Side-by-side comparison of the three lateral-control architectures along the dimensions that are most relevant for an LKA function.	41

Appendix A

Carla and Pycharm IDE Interfacing

To control and communicate with Carla, the Python API is used. Therefore, it is necessary to have a Python version compatible with the Carla version we plan to use. However, updates to Carla often require updating the Python version at regular intervals. This makes it mandatory to reconcile Carla and Python with each new update. However, by taking advantage of uv's ability to use different Python versions through different digital environments, we can solve this problem. Thus, we can have multiple environments and Carla models on the same computer, which also reduces the burden of this process. The steps for setting up Carla-uv communication are as follows:

- Downloaded Carla 0.9.16(UE4)
- Installed Pycharm IDE
- Installed uv python package and project manager:

```
1  set PROJECT=carla_lane
2  powershell -ExecutionPolicy ByPass -c "irm https://astral.sh/uv/
   0.10.8/install.ps1 | iex"
3  uv clean
4  git clone https://github.com/axmud/lane_assist.git %PROJECT%
5  uv init %PROJECT% -p "==3.10.20"
6  cd %PROJECT%
7  uv add setuptools==66.1.1
8  uv add wheel==0.46.3
9  uv add alumentations==0.4.6 --no-build-isolation
10 uv add --requirements requirements.txt
```
- Pytorch requirements. We need to add these lines to the end of pyproject.toml file in the project folder:

```
1  cu121 = ["torch==2.3.1", "torchvision==0.18.1", "torchau-
   dio==2.3.1"]
2  [tool.uv.sources]
3  torch = [
4      { index = "pytorch-cu121", extra = "cu121" }
5  ]
```

```

6  torchvision = [
7      { index = "pytorch-cu121", extra = "cu121" }
8  ]
9  torchaudio = [
10     { index = "pytorch-cu121", extra = "cu121" }
11 ]
12 [[tool.uv.index]]
13 name = "pytorch-cu121"
14 url = "https://download.pytorch.org/whl/cu121"
15 explicit = true

```

- Next, we will run this:

```
1  uv sync --extra cu121
```

- Dependency adjustments:

```

1  uv pip install --no-build-isolation -e .
2  uv pip install numpy==1.23.1
3  uv pip install hydra-core
4  uv pip uninstall setuptools
5  uv pip install setuptools==66.1.1
6  uv pip install carla==0.9.16

```

Appendix B

Code Excerpts

This appendix collects the Python excerpts that support Section 2 and Section 3. The full source tree is available in the project repository; what follows is a curated selection of the routines that are referenced explicitly in the text.

BA Data Collectors (Chapter 2)

The two routines used during the data-gathering phase. The first one, `carla_data_collector`, records the dynamic state of the ego vehicle; the second one, `carla_data_collector_2`, records the geometric reference exposed by the CARLA way-point graph. Both are called once per tick from the main loop.

```
1  def carla_data_collector(vehicle: carla.Vehicle):
2      frame = vehicle.get_world().get_snapshot().frame
3      transform = vehicle.get_transform()
4      location = transform.location
5      rotation = transform.rotation
6      velocity = vehicle.get_velocity()
7      acceleration = vehicle.get_acceleration()
8      angular_velocity = vehicle.get_angular_velocity()
9      waypoint = vehicle.get_world().get_map().get_waypoint(location)
10
11     return {
12         "frame": frame,
13         "location": {"x": location.x, "y": location.y, "z":
location.z},
14         "rotation": {"pitch": rotation.pitch, "yaw": rotation.yaw,
"roll": rotation.roll},
15         "velocity": {"x": velocity.x, "y": velocity.y, "z":
velocity.z},
16         "acceleration": {"x": acceleration.x, "y": acceleration.y,
"z": acceleration.z},
17         "angular_velocity": {"x": angular_velocity.x,
18                               "y": angular_velocity.y,
19                               "z": angular_velocity.z},
```

```

20         "waypoint": {"x": waypoint.transform.location.x,
21                       "y": waypoint.transform.location.y,
22                       "z": waypoint.transform.location.z},
23     }
24
25
26     def carla_data_collector_2(vehicle: carla.Vehicle):
27         waypoint = vehicle.get_world().get_map().get_waypoint(
28             vehicle.get_transform().location)
29         loc = waypoint.transform.location
30         fwd = waypoint.transform.get_forward_vector()
31         return {"x": loc.x, "y": loc.y,
32                "vector_x": fwd.x, "vector_y": fwd.y}

```

The duplicate-suppression block that wraps `carla_data_collector_2` to avoid writing identical rows to the `.csv` file when the closest waypoint does not change between consecutive ticks:

```

1     data = {"waypoint_x": [], "waypoint_y": [],
2             "vector_x": [], "vector_y": []}
3
4     while not crashed:
5         world.tick()
6         sample = carla_data_collector_2(ego_vehicle)
7         if (not data["waypoint_x"]
8             or sample["x"] != data["waypoint_x"][-1]
9             or sample["y"] != data["waypoint_y"][-1]):
10             data["waypoint_x"].append(sample["x"])
11             data["waypoint_y"].append(sample["y"])
12             data["vector_x"].append(sample["vector_x"])
13             data["vector_y"].append(sample["vector_y"])
14
15     pd.DataFrame(data).to_csv("carla_data.csv", index=True)

```

The corresponding logging routine for the manual-driving mode. The autopilot is disengaged with `vehicle.set_autopilot(False)` and the user inputs from the keyboard are translated by the `ControlObject` class (also reported in this appendix) into a `carla.VehicleControl` object that is applied to the ego vehicle.

```

1     data = {
2         "frame": [], "x": [], "y": [], "yaw": [],
3         "v_x": [], "v_y": [], "a_x": [],
4         "throttle": [], "brake": [], "steer": []
5     }

```

```

6
7  while not crashed:
8      world.tick()
9      snapshot      = world.get_snapshot()
10     transform      = ego_vehicle.get_transform()
11     velocity       = ego_vehicle.get_velocity()
12     acceleration    = ego_vehicle.get_acceleration()
13     control        = ego_vehicle.get_control()
14
15     data["frame"].append(snapshot.frame)
16     data["x"].append(transform.location.x)
17     data["y"].append(transform.location.y)
18     data["yaw"].append(transform.rotation.yaw)
19     data["v_x"].append(velocity.x)
20     data["v_y"].append(velocity.y)
21     data["a_x"].append(acceleration.x)
22     data["throttle"].append(control.throttle)
23     data["brake"].append(control.brake)
24     data["steer"].append(control.steer)
25
26     pd.DataFrame(data).to_csv("manual_drive_log.csv", index=False)

```

BB Cross-Track Error (Chapter 3, Section 3.4)

The vectorised implementation of Equation (3.23). A monotonically increasing global index is used so that the search remains $O(1)$ in the length of the reference path and so that the controller cannot “snap” backwards on a closed circuit.

```

1  index = 0
2  FUTURE_HORIZON = 10
3
4  def lane_shift_calculator(vehicle: carla.Vehicle):
5      global index
6      loc = vehicle.get_transform().location
7      car_x, car_y = loc.x, loc.y
8
9      end = min(index + FUTURE_HORIZON + 1, len(wp_x_arr))
10     dx = wp_x_arr[index:end] - car_x
11     dy = wp_y_arr[index:end] - car_y
12     sq_dist = dx * dx + dy * dy

```

```

13     offset = int(np.argmin(sq_dist))
14     nearest_index = index + offset
15     index = nearest_index
16
17     vx = vec_x_arr[nearest_index]
18     vy = vec_y_arr[nearest_index]
19     lane_shift = dx[offset] * vy - dy[offset] * vx
20
21     nearest_dist = float(np.sqrt(sq_dist[offset]))
22     return lane_shift, nearest_dist

```

BC Heading Error and Look-Ahead Target (Chapter 3, Section 3.5)

The heading error is computed via a two-argument arctangent so that all four quadrants are handled correctly. The look-ahead target is the first waypoint on the recorded path whose distance from the vehicle exceeds $L_{d(v)} = L_d^{\min} + k_L v$.

```

1  LOOKAHEAD_MIN = 4.0    # m
2  LOOKAHEAD_K   = 0.6    # s
3
4  def get_lookahead_target(vehicle: carla.Vehicle):
5      loc = vehicle.get_transform().location
6      car_x, car_y = loc.x, loc.y
7      v = vehicle.get_velocity()
8      speed_mps = math.sqrt(v.x ** 2 + v.y ** 2)
9      Ld = LOOKAHEAD_MIN + LOOKAHEAD_K * speed_mps
10     Ld_sq = Ld * Ld
11
12     for j in range(index, len(wp_x_arr)):
13         ddx = wp_x_arr[j] - car_x
14         ddy = wp_y_arr[j] - car_y
15         if ddx * ddx + ddy * ddy >= Ld_sq:
16             return float(wp_x_arr[j]), float(wp_y_arr[j]), j
17     last = len(wp_x_arr) - 1
18     return float(wp_x_arr[last]), float(wp_y_arr[last]), last
19
20
21 def heading_error(vehicle: carla.Vehicle, target_x: float, target_y:
    float):

```

```

22     tf = vehicle.get_transform()
23     fwd = tf.get_forward_vector()
24     v_vec = np.array([fwd.x, fwd.y, 0.0])
25     w_vec = np.array([target_x - tf.location.x,
26                       target_y - tf.location.y, 0.0])
27     norm = np.linalg.norm(v_vec) * np.linalg.norm(w_vec)
28     if norm < 1e-6:
29         return 0.0
30     angle = math.acos(np.clip(np.dot(v_vec, w_vec) / norm, -1.0,
1.0))
31     cross_z = v_vec[0] * w_vec[1] - v_vec[1] * w_vec[0]
32     if cross_z < 0:
33         angle = -angle
34     return angle

```

BD PID Controllers (Chapter 3, Section 3.3)

The discrete PID law of Equation (3.21) is implemented as two stateful controllers, one for the lateral channel and one for the longitudinal channel. The state is held in a fixed-length deque, which provides an implicit anti-windup limit equal to the deque length times the sampling period. The output is then clipped to the actuator range.

```

1  class PIDLateralController:
2      def __init__(self, K_P=1.95, K_I=0.05, K_D=0.2, dt=0.05):
3          self._K_P, self._K_I, self._K_D, self._dt = K_P, K_I, K_D,
dt
4          self._e_buffer = deque(maxlen=10)
5
6      def run_step(self, heading_err):
7          self._e_buffer.append(heading_err)
8          if len(self._e_buffer) >= 2:
9              de = (self._e_buffer[-1] - self._e_buffer[-2]) /
self._dt
10             ie = sum(self._e_buffer) * self._dt
11         else:
12             de = 0.0
13             ie = 0.0
14         return float(np.clip(
15             self._K_P * heading_err + self._K_D * de + self._K_I *
ie,

```



```

16         -1.0, 1.0))
17
18
19 class PIDLongitudinalController:
20     def __init__(self, K_P=0.3, K_I=0.05, K_D=0.0, dt=0.05):
21         self._K_P, self._K_I, self._K_D, self._dt = K_P, K_I, K_D,
dt
22         self._e_buffer = deque(maxlen=10)
23
24     def run_step(self, target_speed, current_speed):
25         error = target_speed - current_speed
26         self._e_buffer.append(error)
27         if len(self._e_buffer) >= 2:
28             de = (self._e_buffer[-1] - self._e_buffer[-2]) /
self._dt
29             ie = sum(self._e_buffer) * self._dt
30         else:
31             de = 0.0
32             ie = 0.0
33         return float(np.clip(
34             self._K_P * error + self._K_D * de + self._K_I * ie,
35             -1.0, 1.0))

```

The combined controller `VehiclePIDController` couples the two channels and returns a single `carla.VehicleControl` object per tick. The signed acceleration produced by the longitudinal controller is mapped to the throttle when positive and to the brake when negative; the steering output of the lateral controller is rate-limited to a maximum increment of 0.1 per step before being applied.

```

1 class VehiclePIDController:
2     def __init__(self, vehicle, args_lateral=None,
args_longitudinal=None,
3         max_throttle=0.75, max_brake=0.3,
max_steering=0.8):
4         self._vehicle = vehicle
5         self._max_throt = max_throttle
6         self._max_brake = max_brake
7         self._max_steer = max_steering
8         self._lat = PIDLateralController(**args_lateral)
9         self._lon = PIDLongitudinalController(**args_longitudinal)
10        self.past_steering = 0.0
11

```

```

12     def run_step_h(self, target_speed, heading_err):
13         v = self._vehicle.get_velocity()
14         current_speed = 3.6 * math.sqrt(v.x**2 + v.y**2 + v.z**2)
15         accel = self._lon.run_step(target_speed, current_speed)
16         steer = self._lat.run_step(heading_err)
17         return self._build_control(accel, steer)
18
19     def _build_control(self, acceleration, steering):
20         control = carla.VehicleControl()
21         if acceleration >= 0.0:
22             control.throttle = min(acceleration, self._max_throt)
23             control.brake = 0.0
24         else:
25             control.throttle = 0.0
26             control.brake = min(abs(acceleration), self._max_brake)
27         if steering > self.past_steering + 0.1:
28             steering = self.past_steering + 0.1
29         elif steering < self.past_steering - 0.1:
30             steering = self.past_steering - 0.1
31         steering = max(-self._max_steer, min(self._max_steer, steering))
32         control.steer = steering
33         self.past_steering = steering
34         return control

```

BE Inverse Perspective Mapping (Chapter 3, Section 3.7)

The `pixel_to_vehicle` routine projects a list of image-plane points onto the ground plane in the body frame of the vehicle, under the pinhole-camera and flat-ground assumptions described in Section 3.7.

```

1     def pixel_to_vehicle(pixel_pts, cam_height,
2                           image_width=1280, image_height=720,
3                           fov_deg=90.0, cam_x_offset=0.0,
4                           max_range=40.0):
5         p = np.asarray(pixel_pts, dtype=np.float64)
6         u, v = p[:, 0], p[:, 1]

```

```

7      fx = fy = image_width / (2.0 * np.tan(np.deg2rad(fov_deg) /
166      2.0))
8      cx, cy = image_width / 2.0, image_height / 2.0
9
10     below_horizon = v > cy + 1e-3
11     u, v = u[below_horizon], v[below_horizon]
12     if u.size == 0:
13         return np.empty((0, 2))
14
15     X_cam = cam_height * fy / (v - cy)
16     Y_cam = X_cam * (u - cx) / fx
17
18     pts = np.column_stack([X_cam + cam_x_offset, Y_cam])
19     return pts[pts[:, 0] <= max_range]

```

The polynomial fit and look-ahead-target evaluation that follow the IPM step:

```

1  def vision_target(centerline_body, Ld, x_min=0.5, x_max=20.0,
166  deg=1):
2      if centerline_body is None or centerline_body.shape[0] < 3:
3          return None
4      pts = centerline_body[(centerline_body[:, 0] > x_min) &
166      (centerline_body[:, 0] < x_max)]
5
6      if pts.shape[0] < 3:
7          return None
8      coeffs = np.polyfit(pts[:, 0], pts[:, 1], deg=deg)
9      y_at_Ld = float(np.polyval(coeffs, Ld))
10     return Ld, y_at_Ld

```

BF System Identification (Chapter 3, Section 3.6)

The two step experiments and the corresponding nonlinear least-squares fits, as implemented in `carla_sysid.py` and `pid_tuning.py`. The longitudinal model is a first-order step response; the lateral identification uses the steady-state value of the yaw rate to back out the steering gain K_{steer} .

```

1  def fit_longitudinal(lon_df, u_step):
2      t = lon_df['t'].to_numpy()
3      v = lon_df['speed_kmh'].to_numpy()
4
5      def model(t, K, tau):
6          return K * u_step * (1.0 - np.exp(-t / tau))

```

```

7
8     K0 = v[-1] / u_step if u_step > 0 else 60.0
9     idx63 = np.argmax(v >= 0.63 * v[-1])
10    tau0 = max(t[idx63], 0.5)
11    popt, _ = curve_fit(model, t, v, p0=[K0, tau0], maxfev=5000)
12    return float(popt[0]), float(popt[1])
13
14
15    def fit_lateral_gain(lat_df, L, delta_cmd):
16        n = len(lat_df)
17        tail = lat_df.iloc[int(0.75 * n):]
18        psi_dot_ss = math.radians(tail['yaw_rate_dps'].mean())
19        V = tail['speed_kmh'].mean() / 3.6
20        K_steer = psi_dot_ss * L / (V * delta_cmd)
21        return float(K_steer), float(V), float(psi_dot_ss)

```

The pole-placement routines that map the identified plant parameters into the analytical PID gains derived in Equation (3.26) and Equation (3.30):

```

1    def tune_lateral_xtrack(K_lat, wn):
2        # double integrator -> (s+wn)^3 -> full PID
3        K_d = 3.0 * wn / K_lat
4        K_p = 3.0 * wn * wn / K_lat
5        K_i = wn * wn * wn / K_lat
6        return {'K_P': K_p, 'K_I': K_i, 'K_D': K_d}
7
8
9    def tune_lateral_heading(K_lat, wn, zeta=1.0):
10        # single integrator -> (s+wn)^2 -> PI sufficient
11        K_p = 2.0 * zeta * wn / K_lat
12        K_i = wn * wn / K_lat
13        return {'K_P': K_p, 'K_I': K_i, 'K_D': 0.0}
14
15
16    def tune_longitudinal(K, tau, wn):
17        # first-order plant -> (s+wn)^2 -> PI
18        K_p = (2.0 * wn * tau - 1.0) / K
19        K_i = (wn * wn * tau) / K
20        return {'K_P': K_p, 'K_I': K_i, 'K_D': 0.0}

```

BG Manual-Override Class

The `ControlObject` class implements the keyboard fallback used in all three architectures. The class follows a two-step design: the `parse_control` method is called for every keyboard event and only updates the internal flags; the heavier `process_control` method is called once per simulation tick and turns those flags into actual `VehicleControl` values, with progressive throttle ramping, automatic reverse engagement at low speed, and exponential return-to-centre on the steering wheel.

```
1  class ControlObject(object):
2      def __init__(self, veh):
3          self._vehicle = veh
4          self._throttle = False
5          self._brake = False
6          self._steer = None
7          self._steer_cache = 0
8          self._control = carla.VehicleControl()
9
10     def parse_control(self, event):
11         if event.type == pygame.KEYDOWN:
12             if event.key == pygame.K_UP:      self._throttle = True
13             if event.key == pygame.K_DOWN:    self._brake = True
14             if event.key == pygame.K_RIGHT:   self._steer = 1
15             if event.key == pygame.K_LEFT:    self._steer = -1
16         if event.type == pygame.KEYUP:
17             if event.key == pygame.K_UP:      self._throttle = False
18             if event.key == pygame.K_DOWN:
19                 self._brake = False
20                 self._control.reverse = False
21             if event.key in (pygame.K_LEFT, pygame.K_RIGHT):
22                 self._steer = None
23
24     def process_control(self):
25         # Throttle / brake combinations
26         if self._throttle:
27             self._control.throttle = min(self._control.throttle +
0.01, 1)
28             self._control.gear = 1
29             self._control.brake = False
30         elif not self._brake:
31             self._control.throttle = 0.0
```

```

32         if self._brake:
33             if (self._vehicle.get_velocity().length() < 0.01
34                 and not self._control.reverse):
35                 self._control.brake = 0.0
36                 self._control.gear = 1
37                 self._control.reverse = True
38                 self._control.throttle = min(self._control.throttle
+ 0.1, 1)
39             elif self._control.reverse:
40                 self._control.throttle = min(self._control.throttle
+ 0.1, 1)
41             else:
42                 self._control.throttle = 0.0
43                 self._control.brake = min(self._control.brake + 0.3,
1)
44         else:
45             self._control.brake = 0.0
46
47         # Steering with low-pass return-to-centre
48         if self._steer is not None:
49             self._steer_cache += 0.03 * self._steer
50             self._steer_cache = max(-0.7, min(0.7,
self._steer_cache))
51             self._control.steer = round(self._steer_cache, 1)
52         else:
53             self._steer_cache *= 0.2
54             if abs(self._steer_cache) < 0.01:
55                 self._steer_cache = 0.0
56             self._control.steer = round(self._steer_cache, 1)
57
58         self._vehicle.apply_control(self._control)

```